



CNN(Convolutional Neural Network)

이해와 활용

목 차

01

오리엔테이션

02

데이터와 차원

03

이미지 데이터셋

03

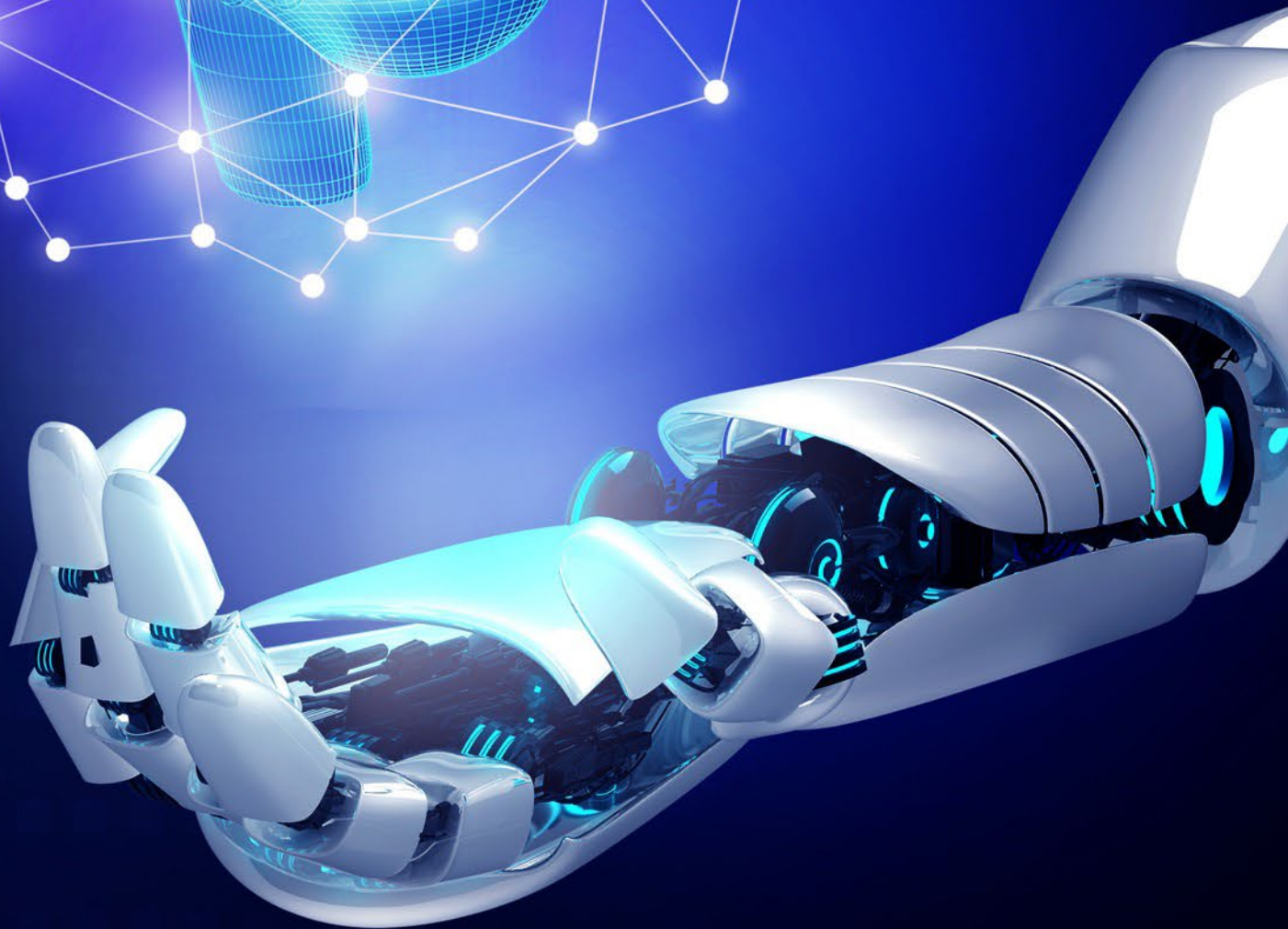
CNN 핵심기술

03

CNN 예시

03

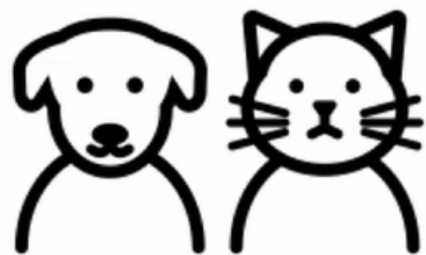
CNN 실습





01. 오리엔테이션

■ 이미지 분류



이미지 분류기



01. 오리엔테이션

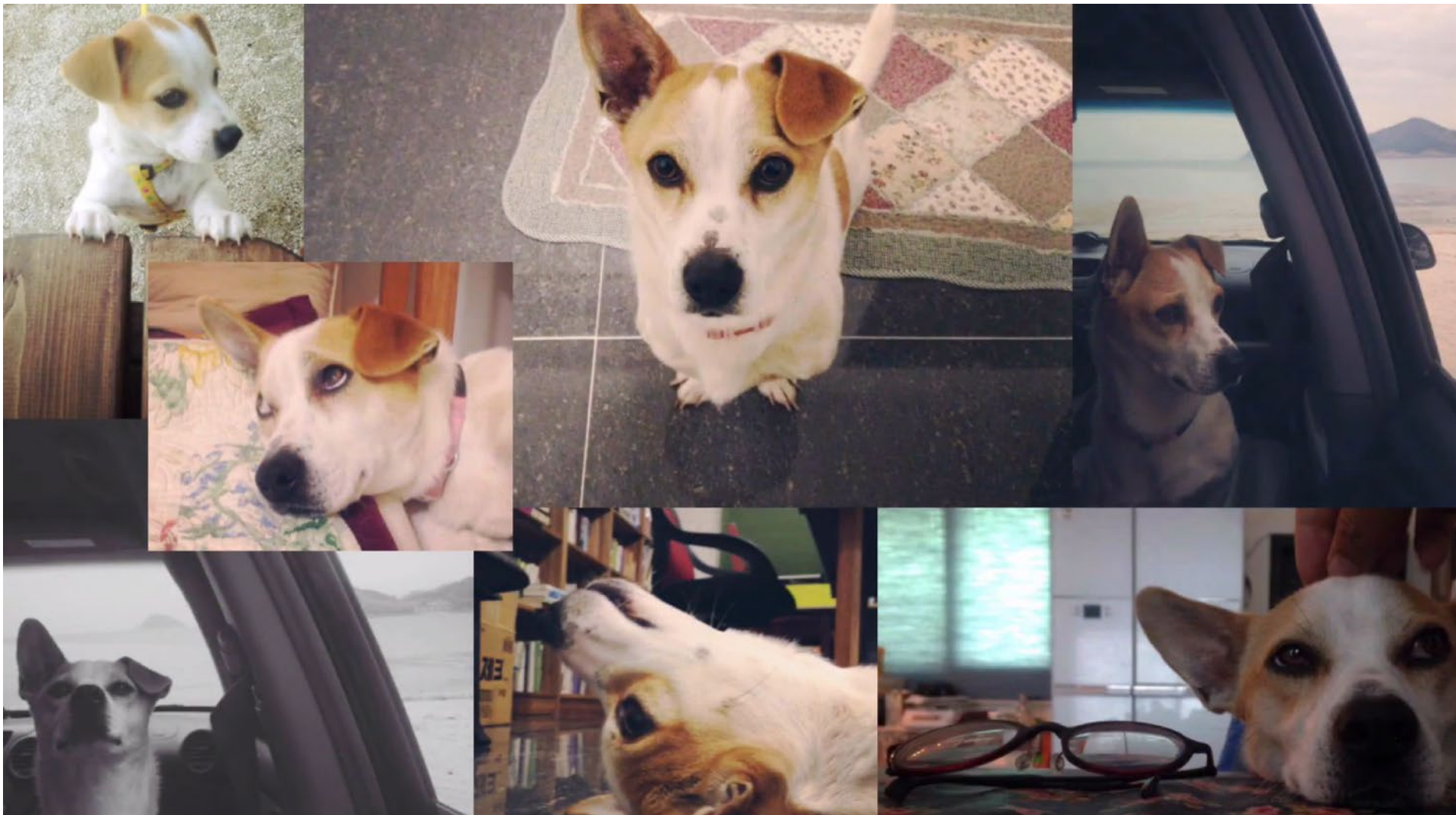
■ 이미지 분류





01. 오리엔테이션

■ 이미지 분류





01. 오리엔테이션

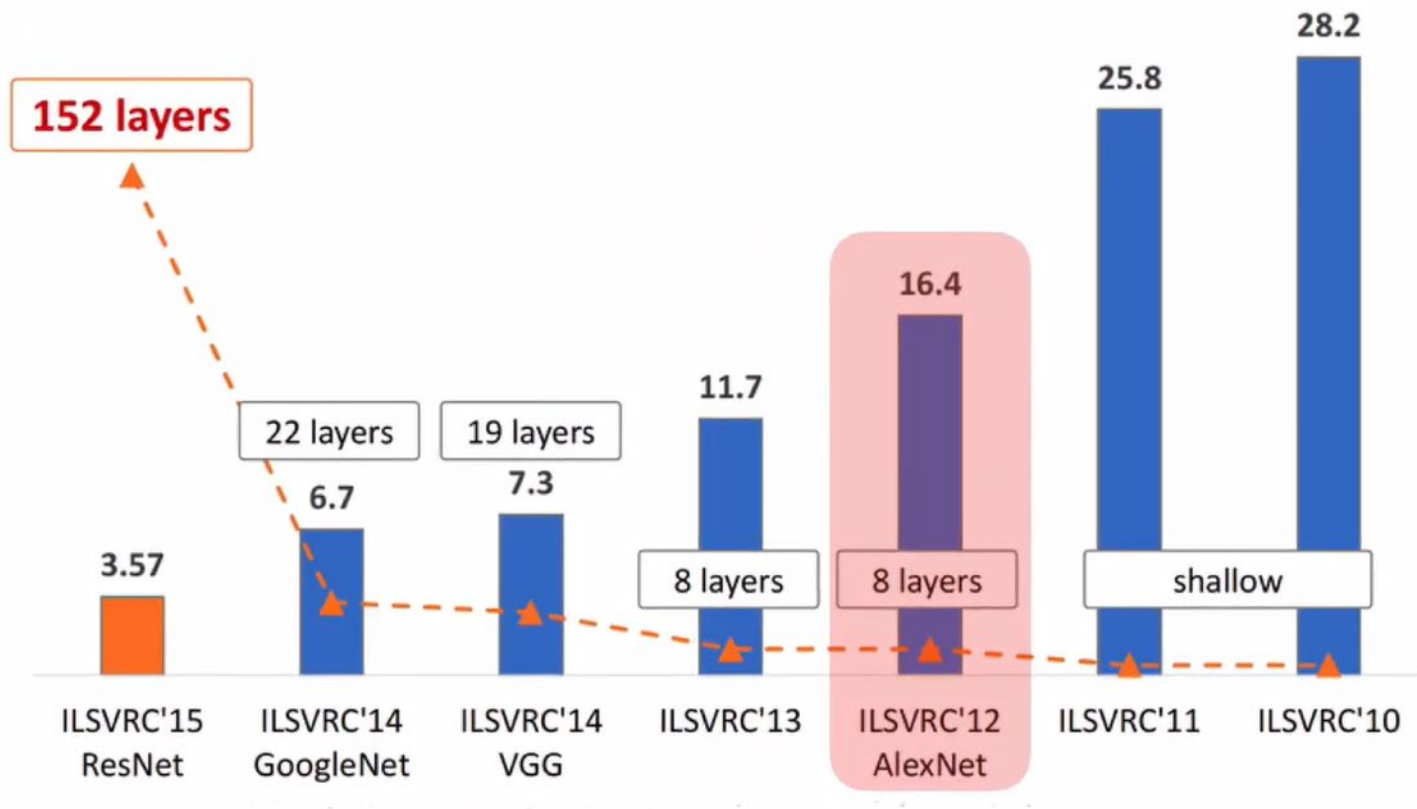
■ 이미지 분류





01. 오리엔테이션

■ 이미지 분류

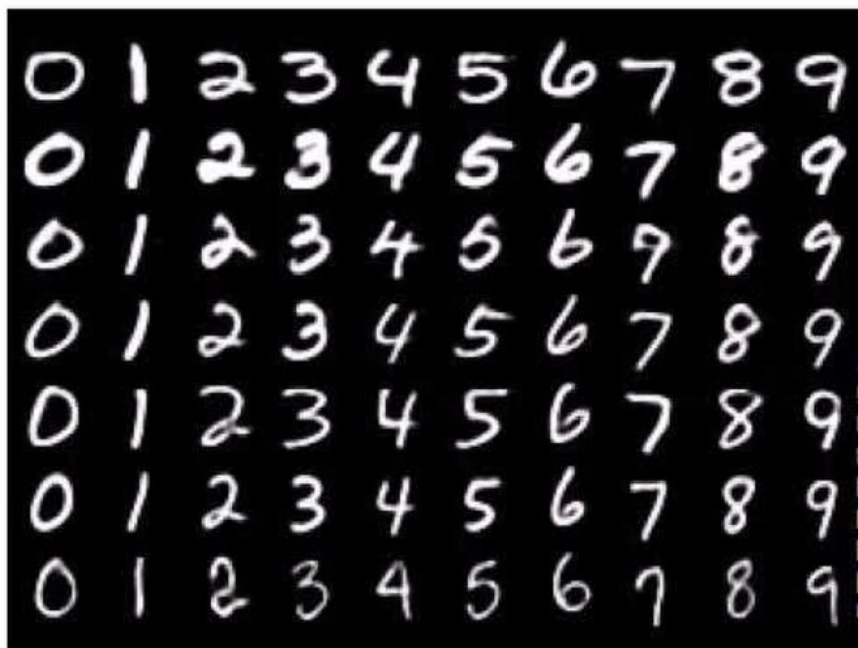




01. 오리엔테이션

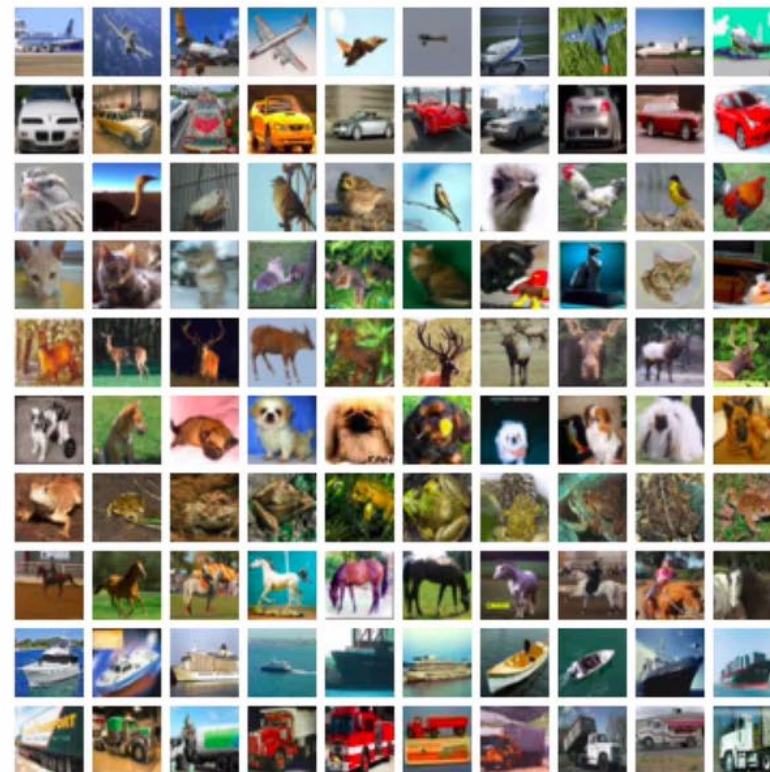
■ 이미지 데이터

MNIST



CIFAR10

airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck





01. 오리엔테이션

■ 이미지 데이터

MNIST(Modified National Institute of Standards and Technology database)

항목	설명
데이터 개수	총 70,000장 (훈련용 60,000장 + 테스트용 10,000장)
이미지 크기	28x28 픽셀, 흑백 이미지 (1채널)
분류 대상	숫자 0 ~ 9 (총 10개 클래스)
용도	딥러닝, 머신러닝 입문 실습용으로 많이 사용됨

CIFAR-10(Canadian Institute for Advanced Research - 10 classes)

항목	설명
데이터 개수	총 60,000장 (훈련용 50,000장 + 테스트용 10,000장)
이미지 크기	32x32 픽셀, 컬러 이미지 (3채널)
분류 대상	총 10개 클래스
클래스 목록	비행기, 자동차, 새, 고양이, 사슴, 개, 개구리, 말, 배, 트럭



02. 데이터와 차원

■ 차원

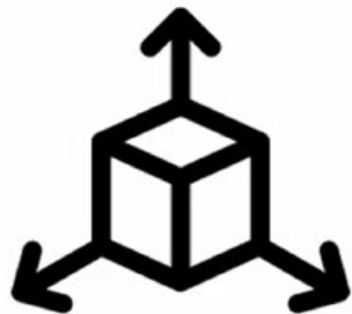




02. 데이터와 차원

■ 차원

차원



표의 열 vs 포함 관계



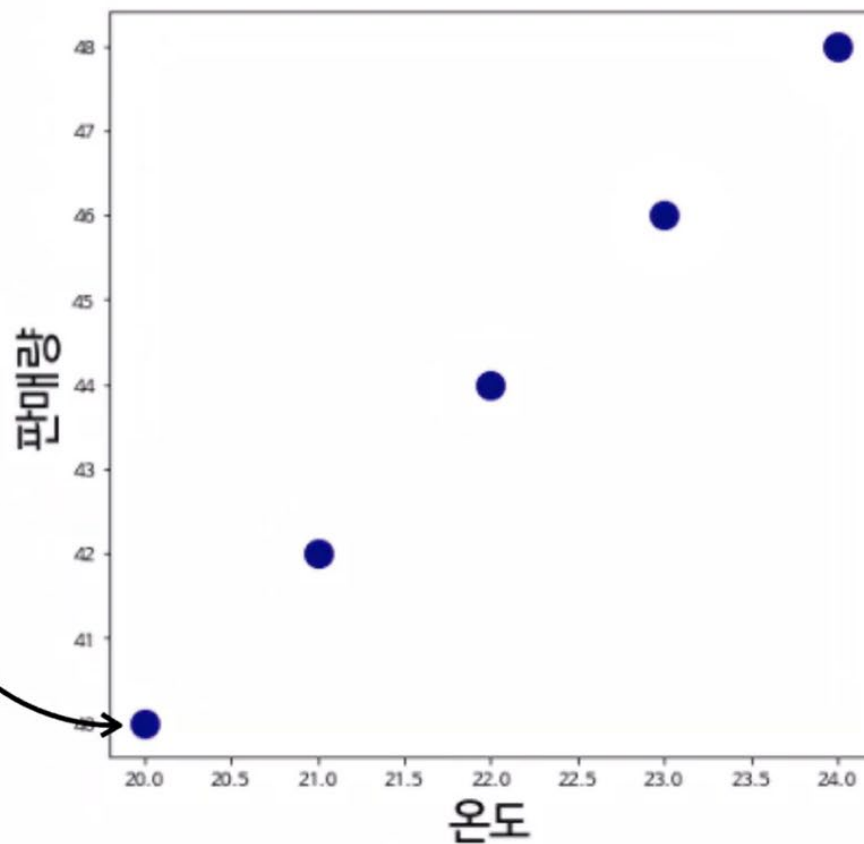
02. 데이터와 차원

■ 차원

표의 열 vs 포함 관계

5

온도	판매량
20	40
21	42
22	44
23	46
24	48





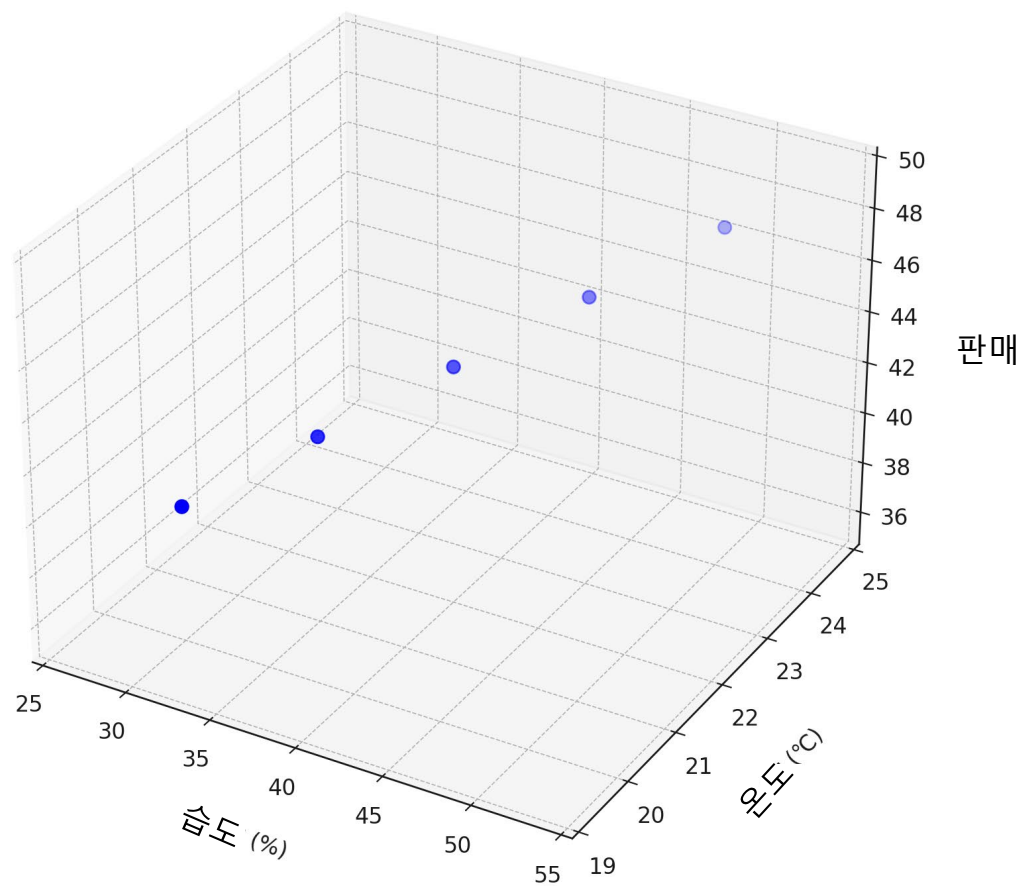
02. 데이터와 차원

■ 차원

표의 열 vs 포함 관계

5

습도	온도	판매
30	20	40
35	21	42
40	22	44
45	23	46
50	24	48





02. 데이터와 차원

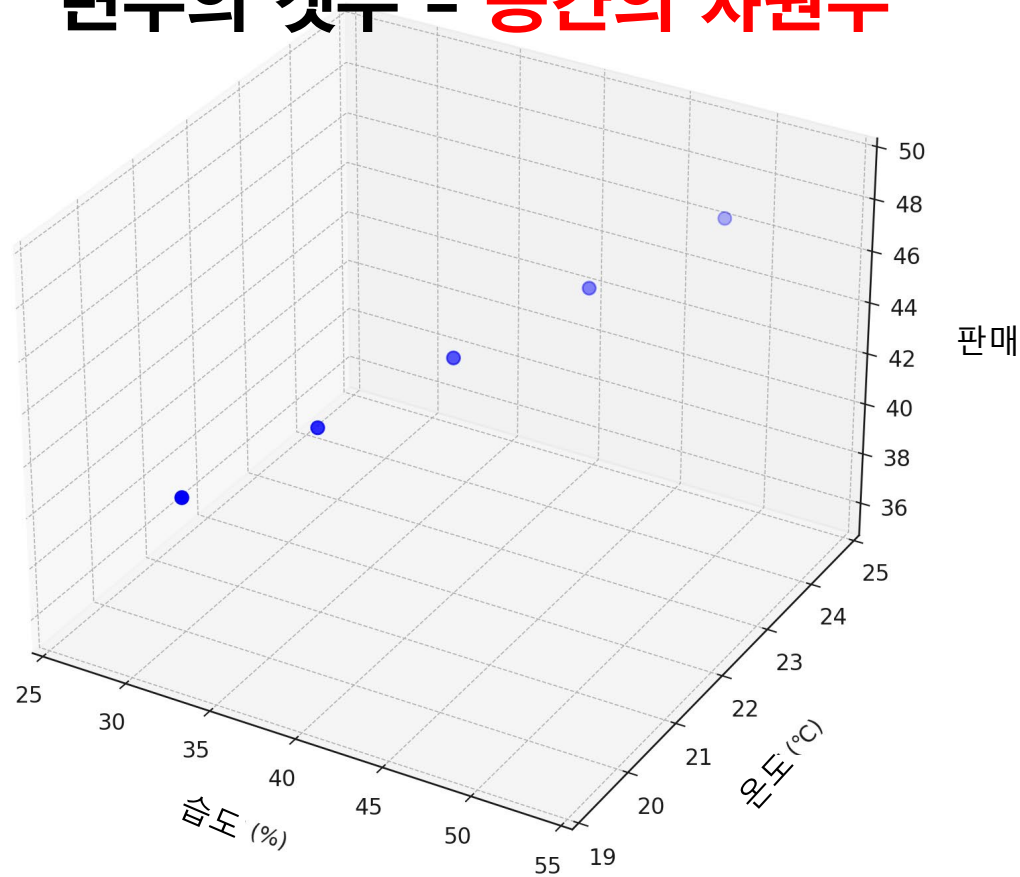
■ 차원

표의 열 vs 포함 관계

습도	온도	판매	...	N
30	20	40	...	
35	21	42	...	
40	22	44	...	
45	23	46	...	
50	24	48	...	

관측치 = **N차원 공간의 한점**

변수의 갯수 = **공간의 차원수**





02. 데이터와 차원

■ 차원

표의 열 vs 포함 관계

꽃잎길이	꽃잎폭	꽃받침길이	꽃받침폭
5.1	3.5	1.4	0.2
4.9	3.0	1.5	0.2
4.7	3.2	1.3	0.2

4차원 공간에 표현되는 관측치

배열의 깊이 = “차원수”

```
x1 = np.array( [5.1, 3.5, 1.4, 0.2] )  
print(x1.ndim, x1.shape)  
# 1차원 형태, (4, )
```

```
x2 = np.array( [4.9, 3.0, 1.4, 0.2] )  
x3 = np.array( [4.7, 3.2, 1.3, 0.2] )
```

```
아이리스 = np.array( [x1, x2, x3] )  
print( 아이리스.ndim, 아이리스.shape )  
# 2차원 형태, (3, 4)
```



02. 데이터와 차원

■ 차원

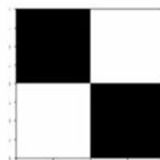
표의 열 vs 포함 관계

```
x1 = np.array( [5.1, 3.5, 1.4, 0.2] )  
print(x1.ndim, x1.shape)  
# 1차원 형태, (4, )
```

```
x2 = np.array( [4.9, 3.0, 1.4, 0.2] )  
x3 = np.array( [4.7, 3.2, 1.3, 0.2] )
```

```
아이리스 = np.array( [x1, x2, x3] )  
print( 아이리스.ndim, 아이리스.shape )  
# 2차원 형태, (3, 4)
```

```
img1 = np.array( [  
    [ 0, 255],  
    [255, 0]  
)  
print(img1.ndim, img1.shape)  
# 2차원 형태, (2, 2)
```



```
img2 = np.array( [[255, 255], [255, 255]] )  
img3 = np.array( [[0, 0], [0, 0]] )
```

```
이미지셋 = np.array( [img1, img2, img3] )  
print( 이미지셋.ndim, 이미지셋.shape )  
# 3차원 형태, (3, 2, 2)
```

4차원 공간에 표현되는 관측치



02. 데이터와 차원

■ 차원

표의 열 vs 포함 관계

```
d1 = np.array( [1, 2, 3] )  
print( d1.ndim, d1.shape )  
# 1차원 형태, (3, )
```

```
d3 = np.array( [d2, d2, d2, d2] )  
print( d3.ndim, d3.shape )  
# 3차원 형태, (4, 5, 3)
```

```
d2 = np.array( [d1, d1, d1, d1, d1] )  
print( d2.ndim, d2.shape )  
# 2차원 형태, (5, 3)
```

```
d4 = np.array( [d3, d3] )  
print( d4.ndim, d4.shape )  
# 4차원 형태, (2, 4, 5, 3)
```

배열의 깊이 = 차원수



02. 데이터와 차원

■ 차원

표의 열 vs 포함 관계

tensor

't'
'e'
'n'
's'
'o'
'r'

tensor of dimensions [6]
(vector of dimension 6)

3	1	4	1
5	9	2	6
5	3	5	8
9	7	9	3
2	3	8	4
6	2	6	4

tensor of dimensions [6,4]
(matrix 6 by 4)

2	1	8	8	8
2	8	5	0	5
2	3	3	0	8
7	7	1	5	6

tensor of dimensions [4,4,2]

```
d2 = np.array( [d1, d1, d1, d1, d1] )  
print( d2.ndim, d2.shape )  
# 2차원 형태, (5, 3)
```

```
d4 = np.array( [d3, d3] )  
print( d4.ndim, d4.shape )  
# 4차원 형태, (2, 4, 5, 3)
```

배열의 깊이 = 차원수



02. 데이터와 차원

■ 차원

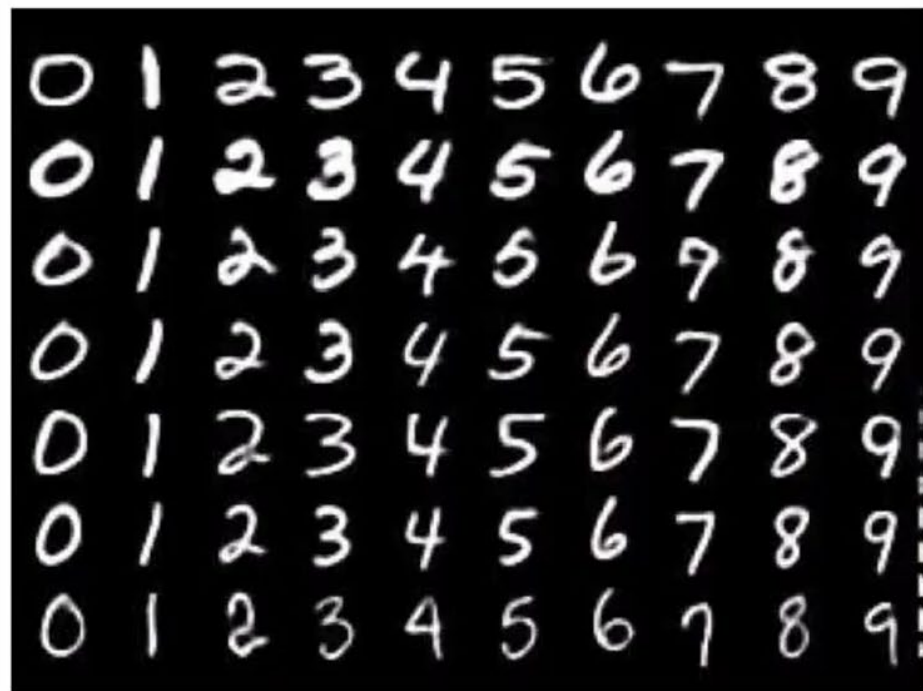
- 데이터 **공간**의 맥락
차원수 = **변수의 개수**
- 데이터 **형태**의 맥락
차원수 = **배열의 깊이**



03. 이미지 데이터셋

■ 이미지 데이터셋

MNIST



CIFAR10

airplane

automobile

bird

cat

deer

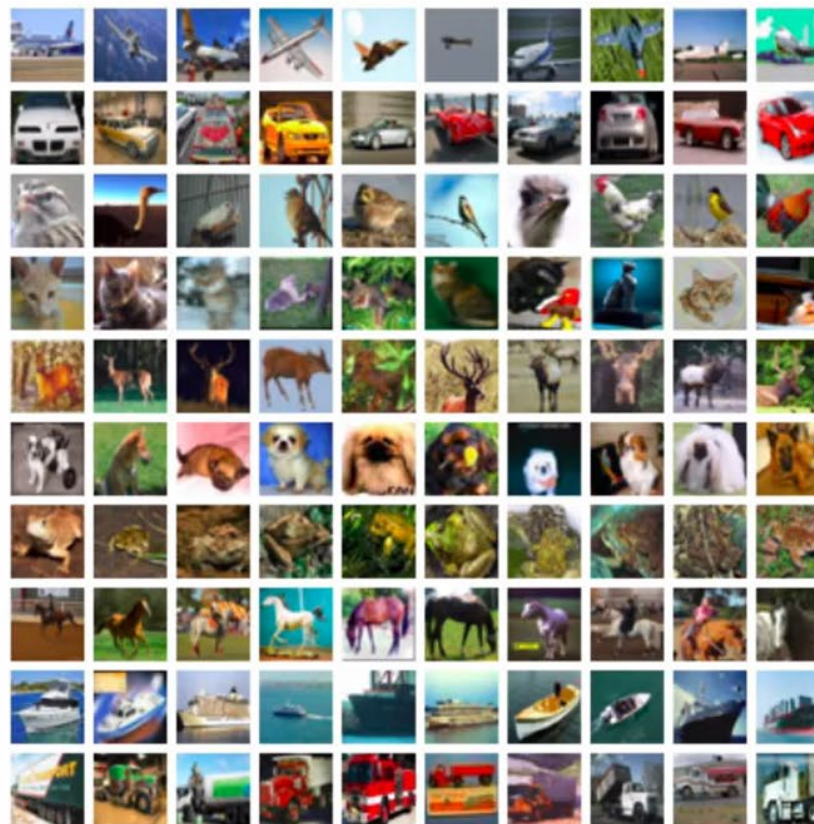
dog

frog

horse

ship

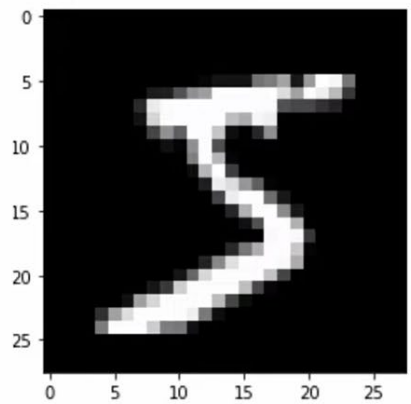
truck



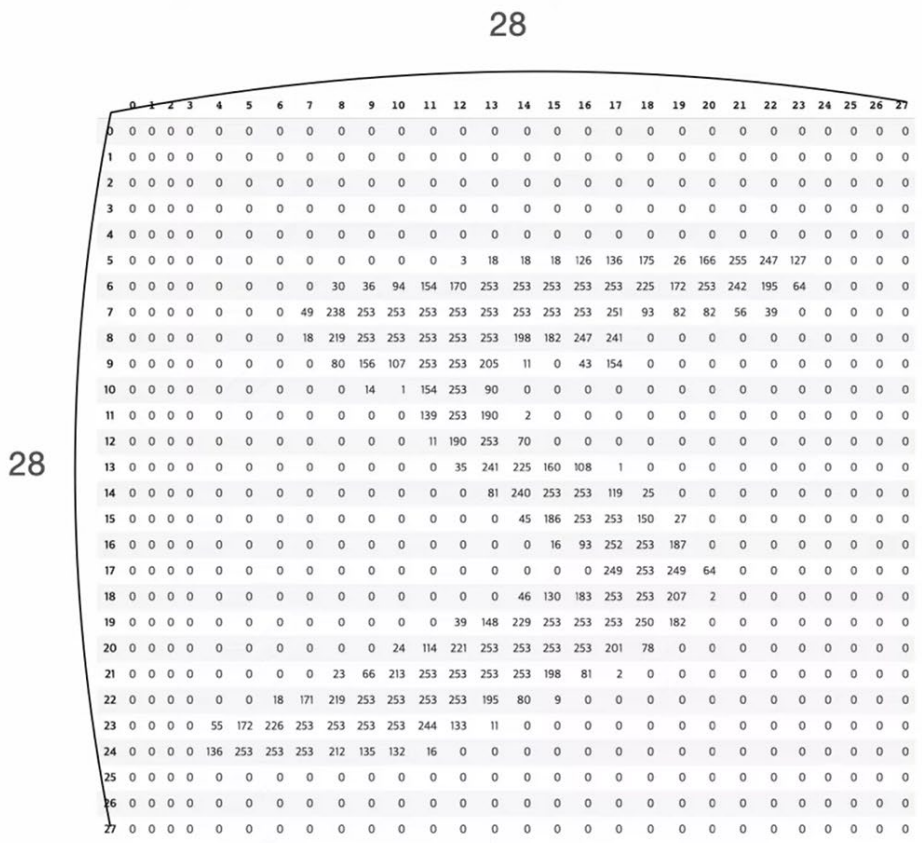


03. 이미지 데이터셋

■ MNIST



개별 Mnist - (28, 28)

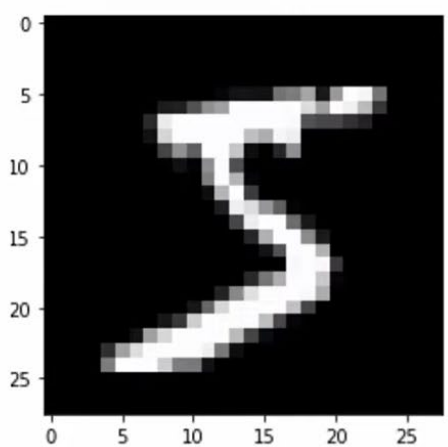


784개의 숫자 (= 28 * 28)



03. 이미지 데이터셋

■ MNIST



개별 Mnist - (28, 28)

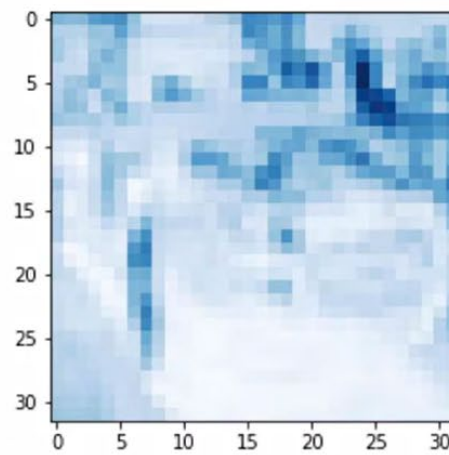
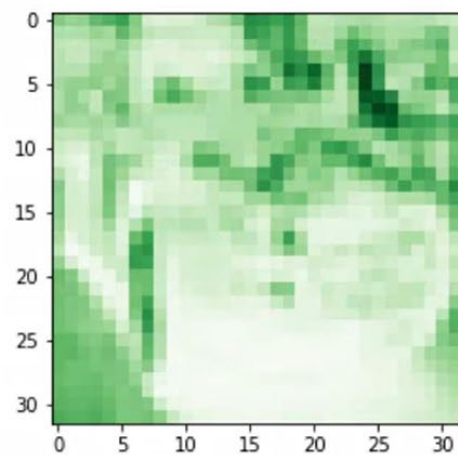
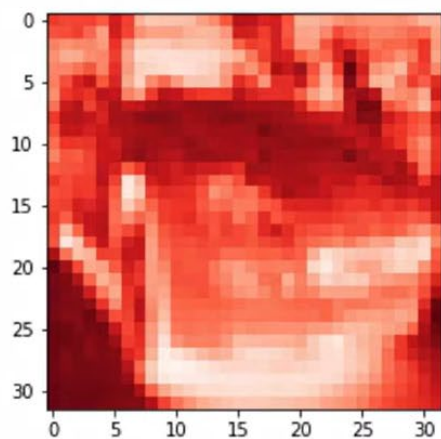
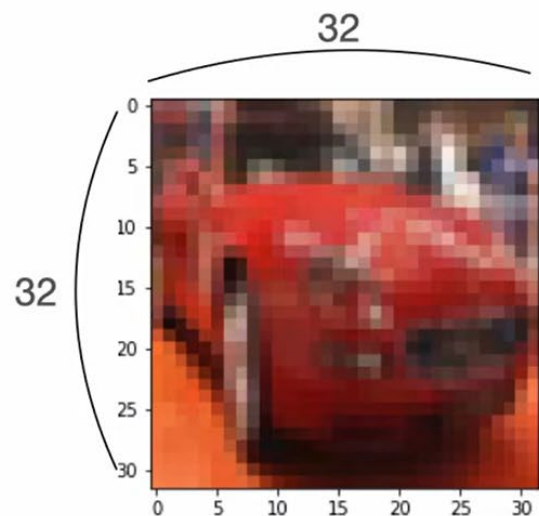
Mnist 셋: (60000, 28, 28)





03. 이미지 데이터셋

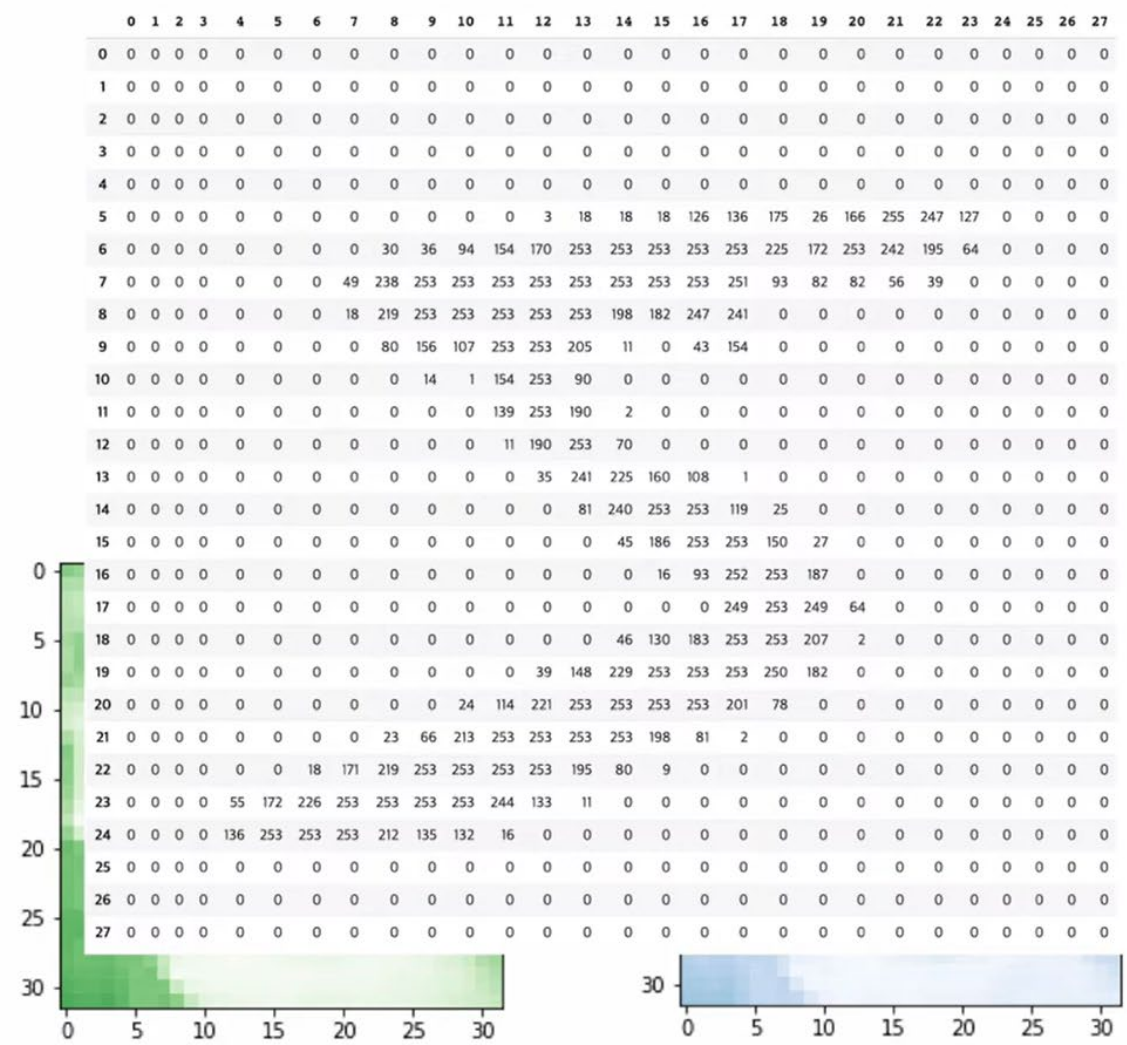
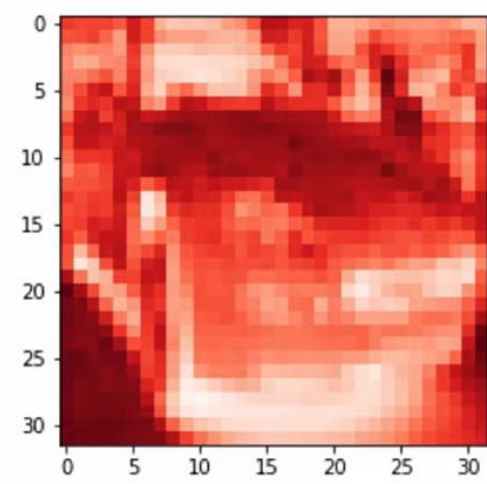
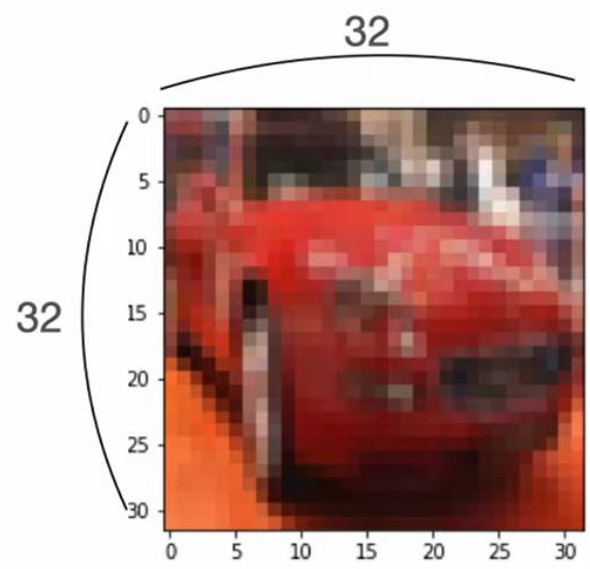
■ CIFAR 10





03. 이미지 데이터셋

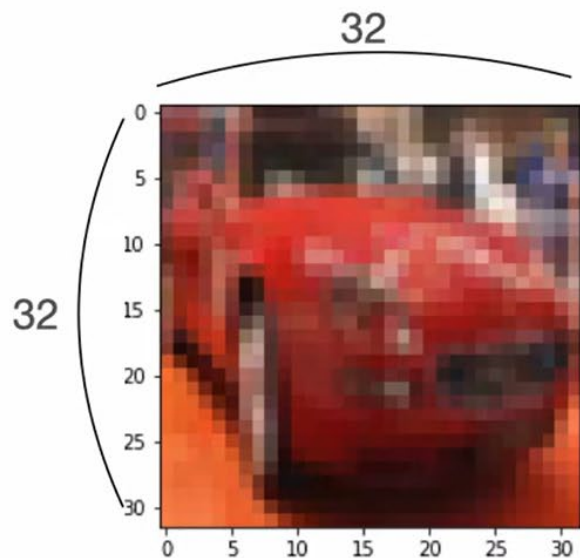
■ CIFAR 10





03. 이미지 데이터셋

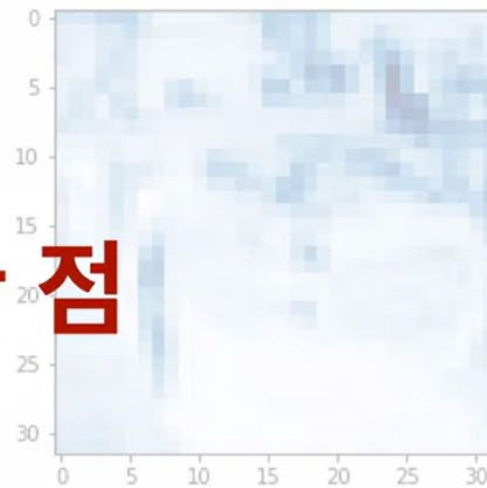
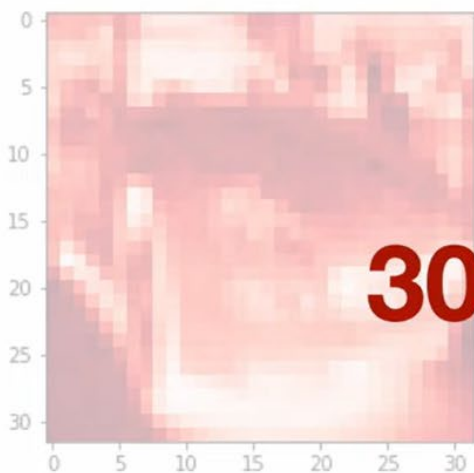
■ CIFAR 10



개별 cifar10 - (32, 32, 3)

cifar10 셋: (5000, 32, 32, 3)

3072개의 숫자 (= $32 * 32 * 3$)



3차원 형태
3072차원 공간의 한 점



03. 이미지 데이터셋

■ CIFAR 10



$$2,448 * 3,264 * 3 = 23,970,816$$



04. CNN핵심 기술-Flatten 총

■ 이미지 분류

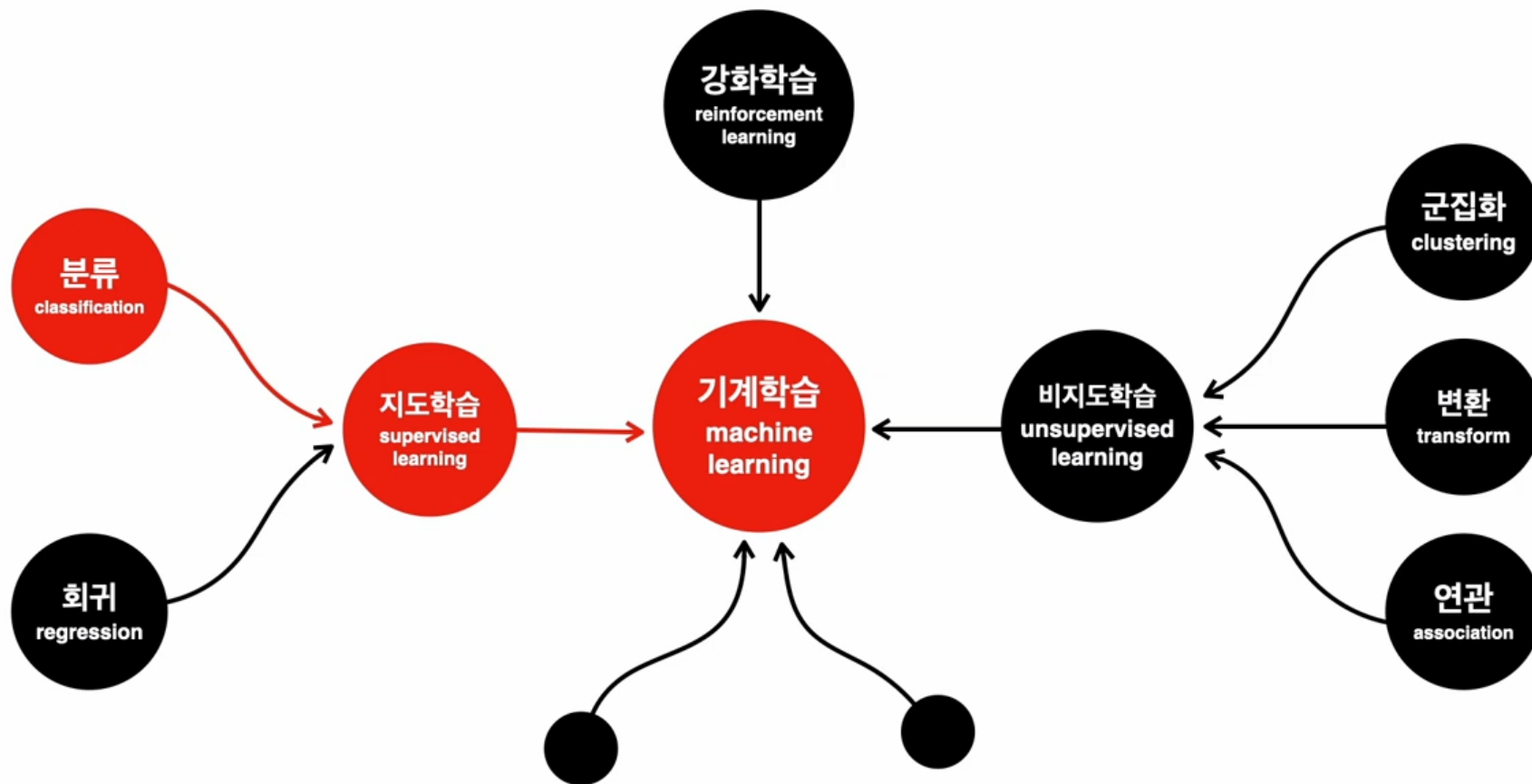


이미지 분류기



04. CNN핵심 기술-Flatten 총

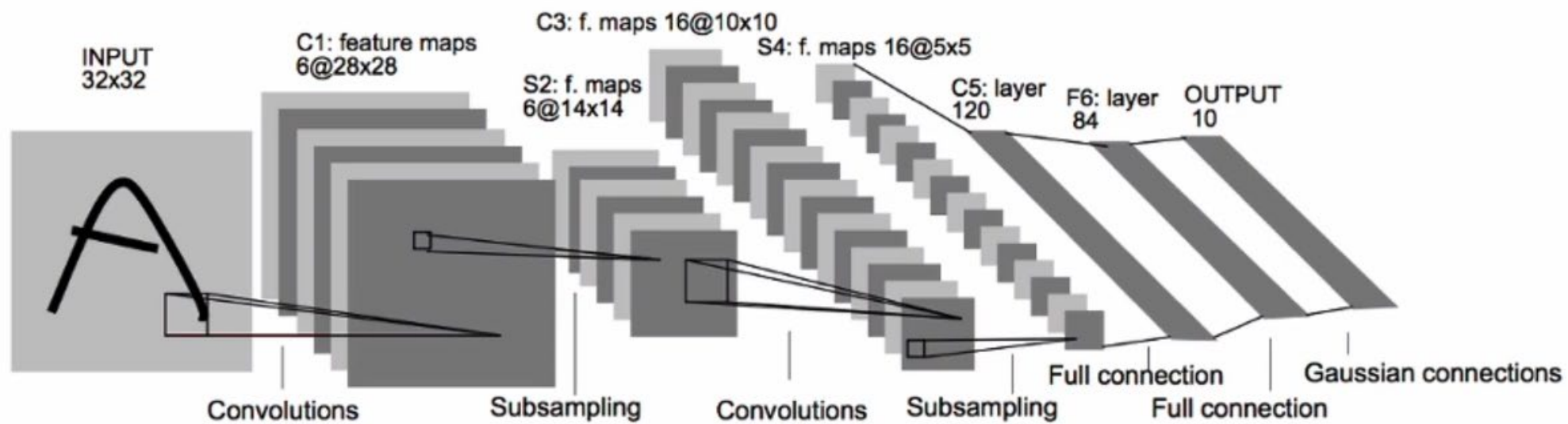
■ 이미지 분류





04. CNN핵심 기술-Flatten 총

■ 이미지 분류기 예시

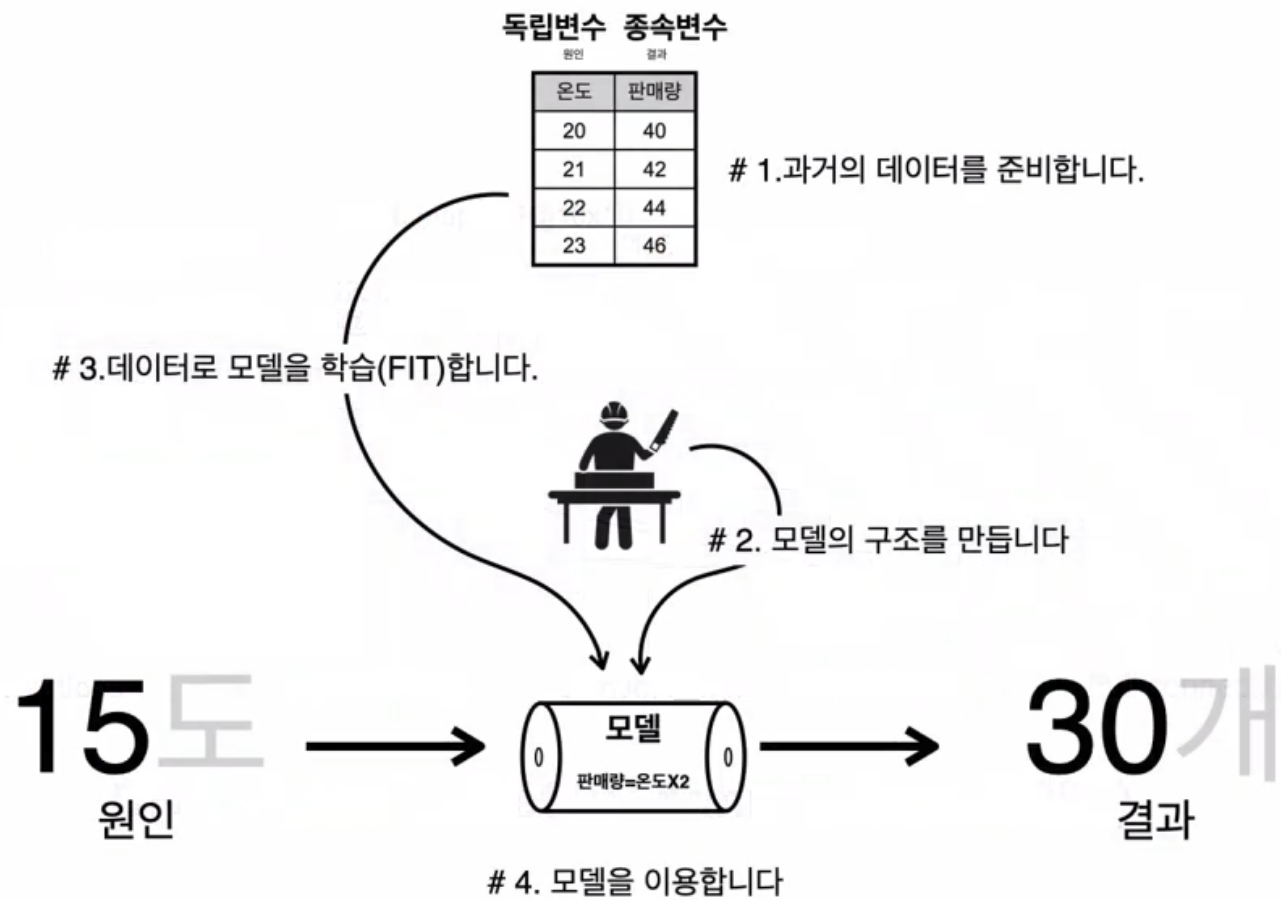


CNN called LeNet by Yann LeCun (1998)



04. CNN핵심 기술-Flatten 총

■ 모델링





04. CNN핵심 기술-Flatten 총

■ reshape

28

```
[ [ 1, 2, 3, ... 28],  
  [ 29, 30, 31, ... 56],  
  ...  
  [757, 758, 759, ... 784],  
]
```

28

784

1	2	3	...	28	29	30	31	...	56		...		757	758	759	...	784

```
print(독립.shape)
```

```
# (60000, 28, 28)
```

```
독립.reshape(60000, 784)
```

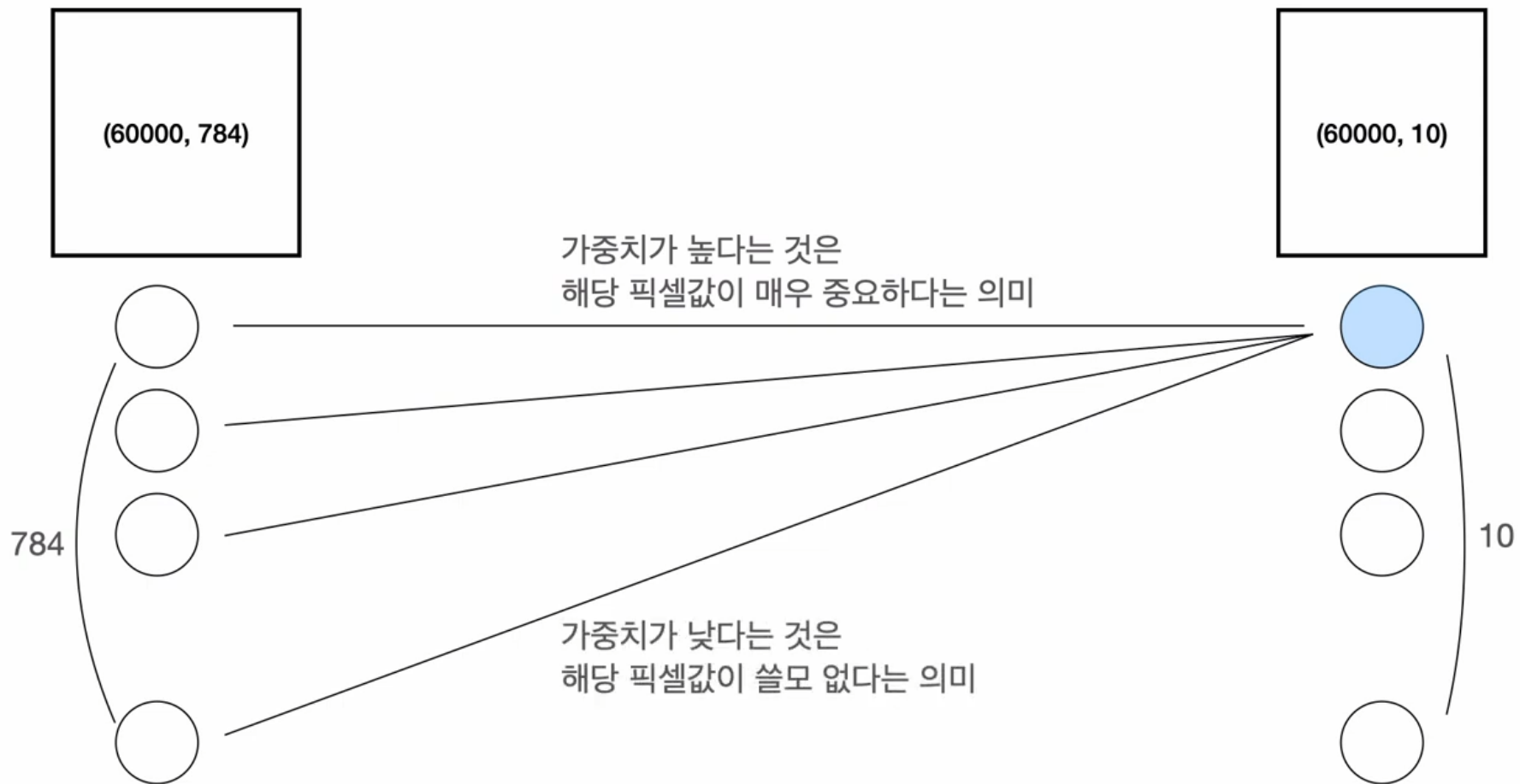
```
print(독립.shape)
```

```
# (60000, 784)
```



04. CNN핵심 기술-Flatten 총

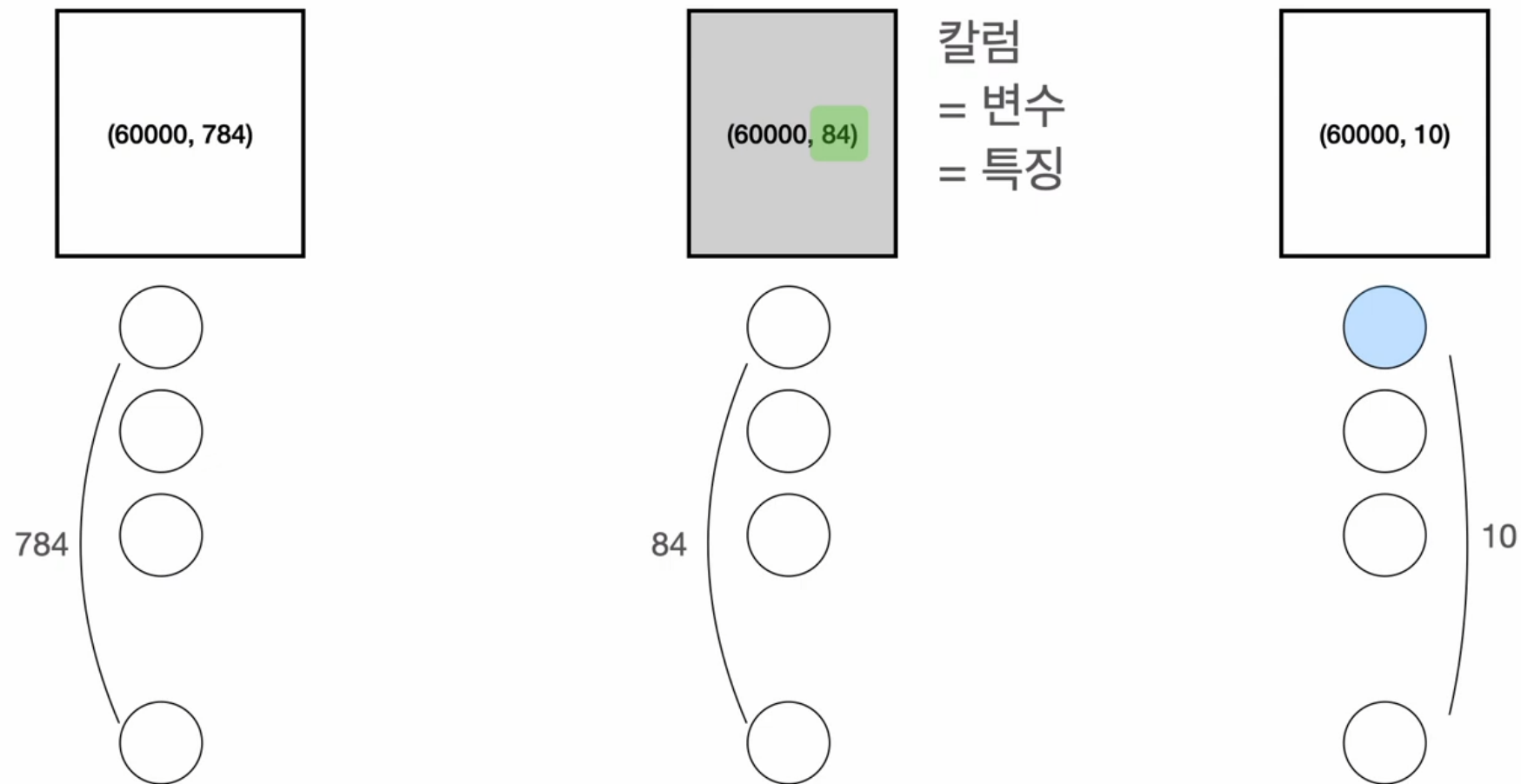
가중치





04. CNN핵심 기술-Flatten 총

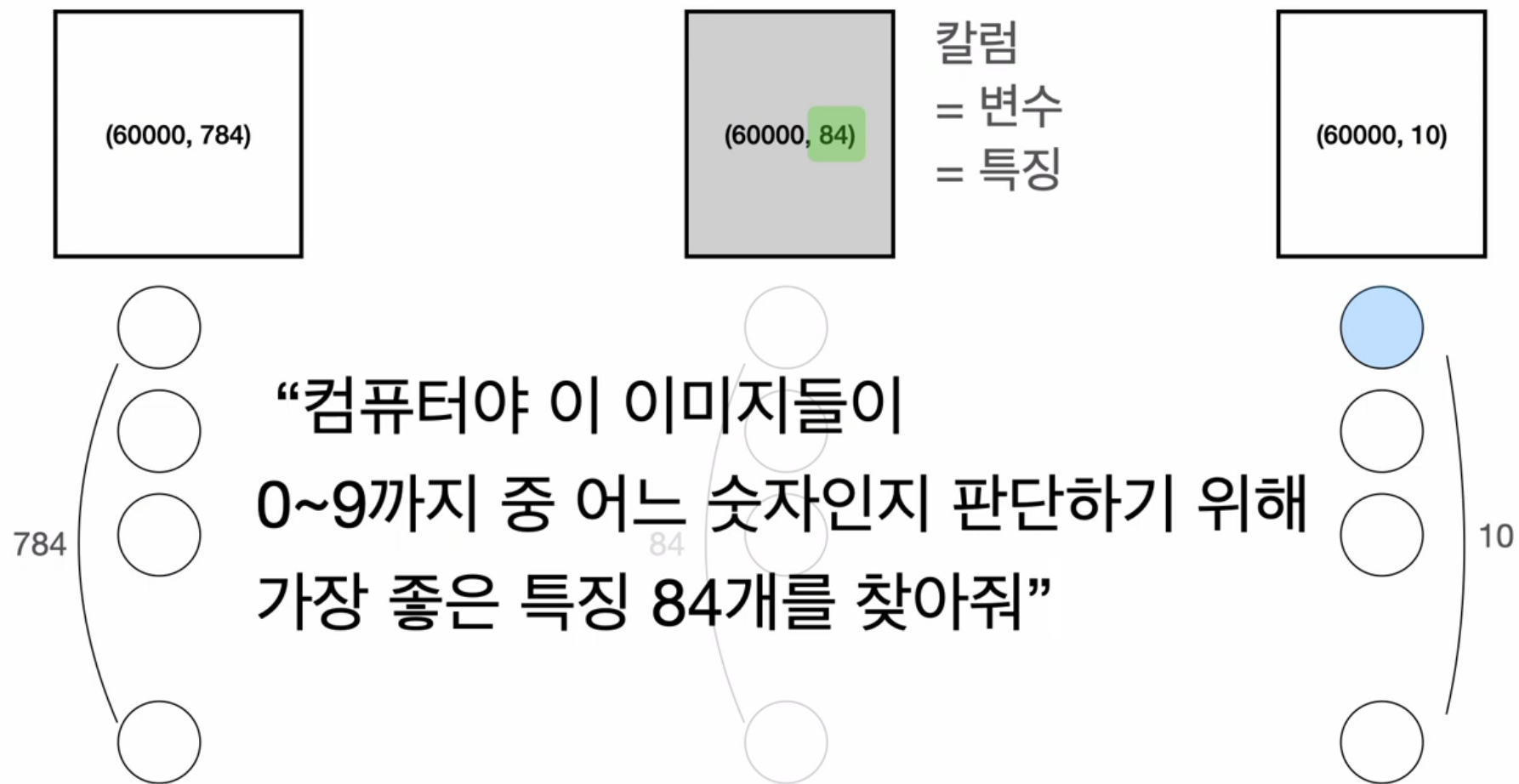
■ 특징추출





04. CNN핵심 기술-Flatten 총

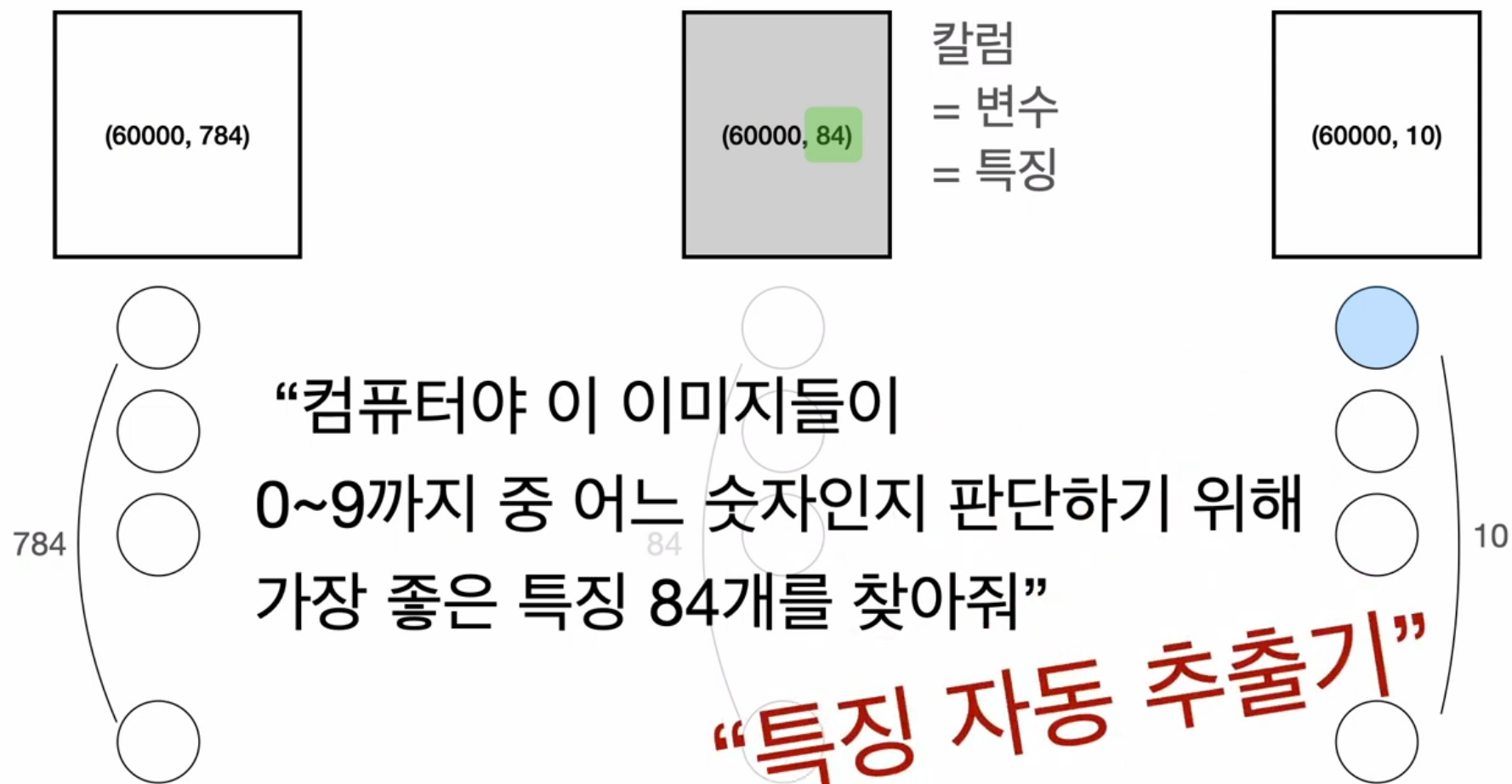
■ 특징추출





04. CNN핵심 기술-Flatten 총

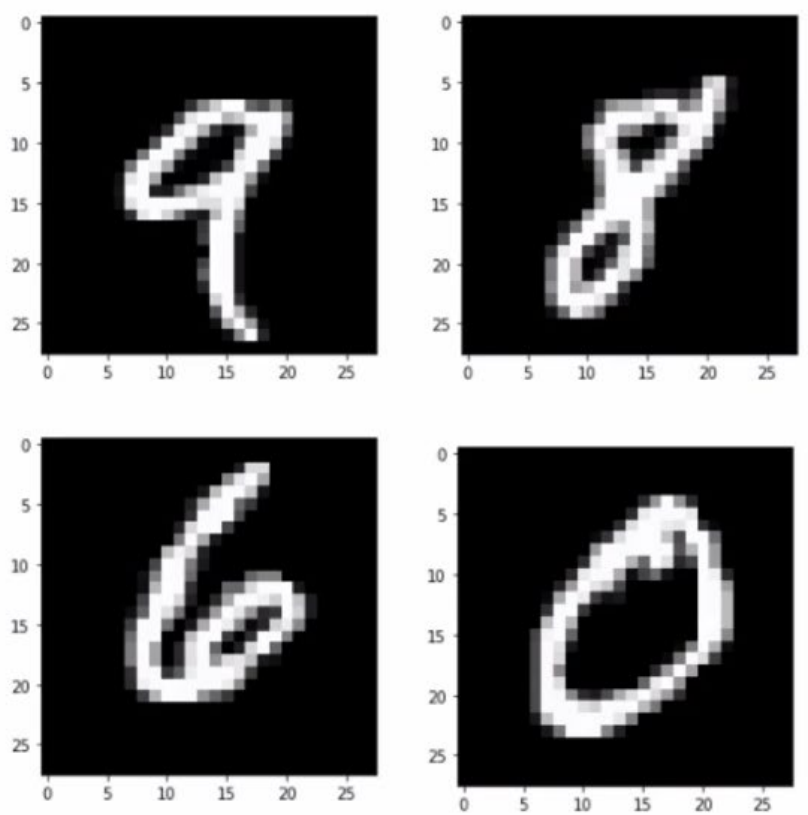
■ 특징추출





04. CNN핵심 기술-Conv2D 총

■ Convolution



숫자	전체	위	아래
9	1	1	0
8	2	1	1
6	1	0	1
0	1	0	0

특정한 패턴의 특징이
어디서 나타나는지를 확인하는 도구
“Convolution”

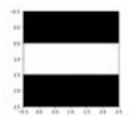
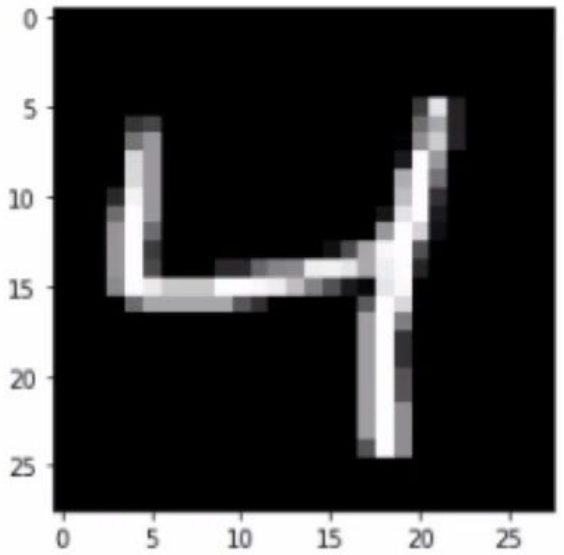


04. CNN핵심 기술-Conv2D 총

■ Convolution

특정한 패턴의 특징이
어디서 나타나는지를 확인하는 도구

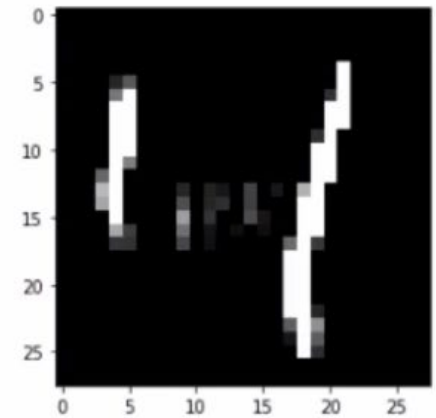
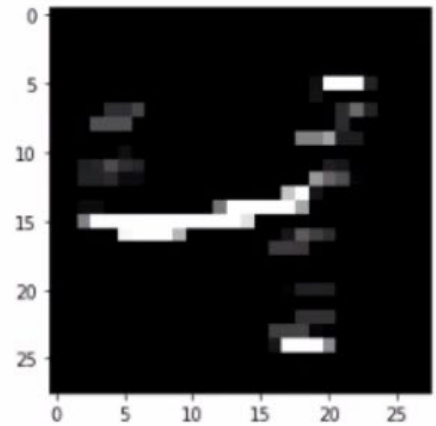
“Convolution”



필터



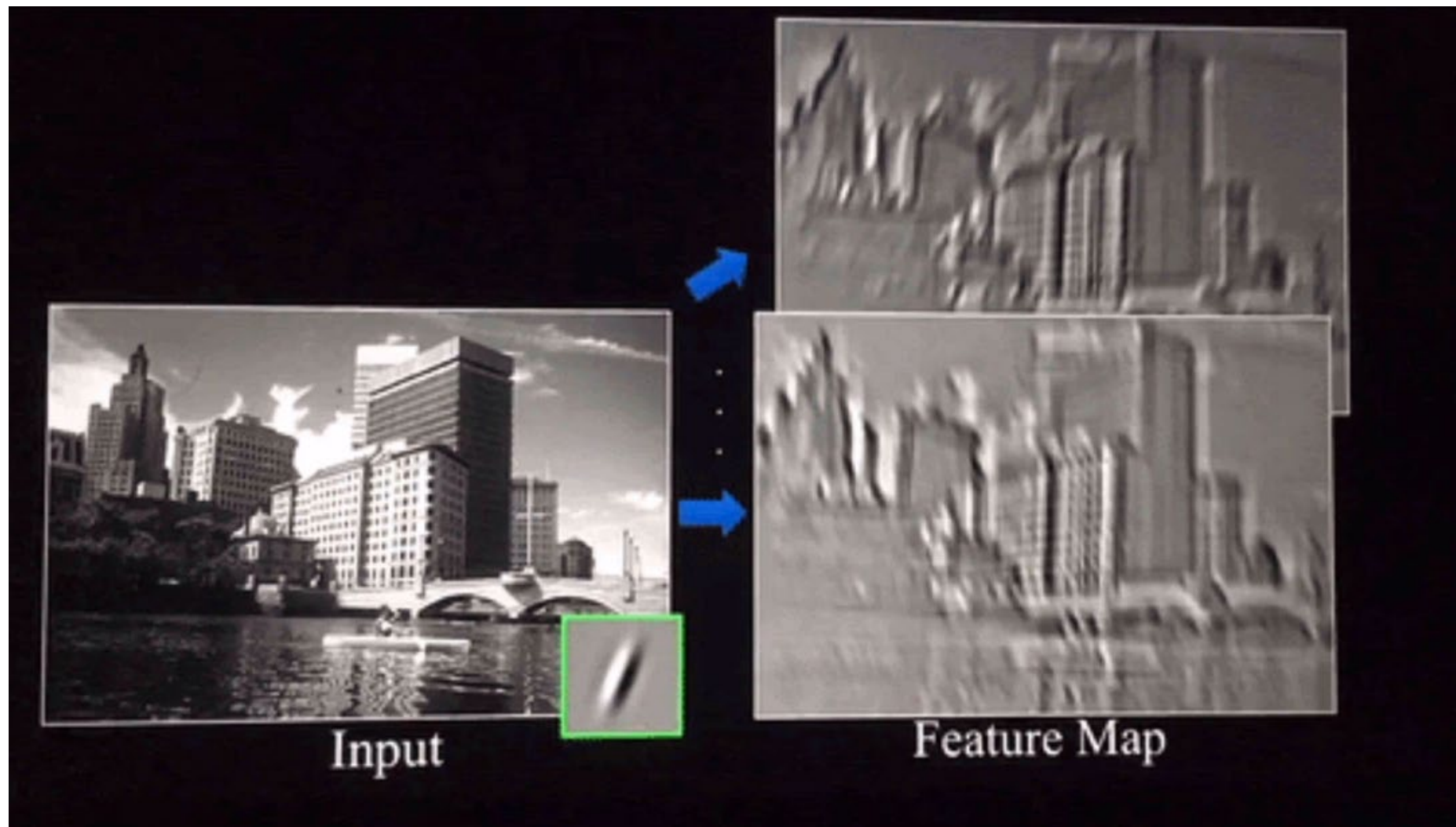
특징맵 feature map





04. CNN핵심 기술-Conv2D 총

■ 머신러닝(Machine learning)





04. CNN핵심 기술-filter(필터)

■ filter(필터)

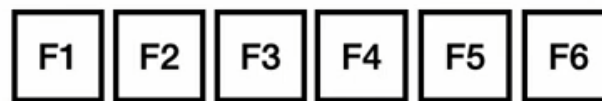
필터의 이해

1. 필터셋은 3차원 형태로 된 **가중치**의 모음
2. 필터셋 하나는 앞선 레이어의 결과인 **“특징맵”** 전체를 본다.
3. **필터셋 개수 만큼** 특징맵을 만든다.

```
model = nn.Sequential( nn.Conv2d(in_channels=1, out_channels=3, kernel_size=5),  
nn.SiLU(),  
nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5), nn.SiLU(), )
```



(3, ~~3~~, 5)



(6, ~~3~~, 5)



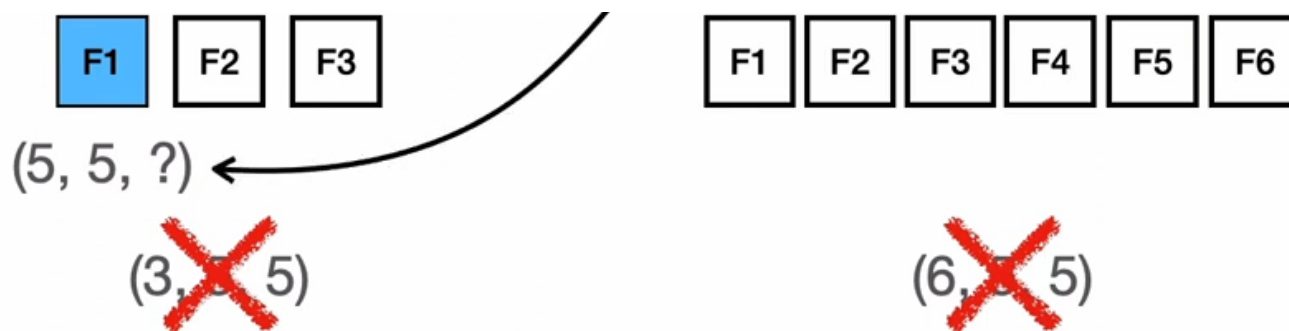
04. CNN핵심 기술-filter(필터)

■ filter(필터)

필터의 이해

1. 필터셋은 3차원 형태로 된 **가중치**의 모음
2. 필터셋 하나는 앞선 레이어의 결과인 **“특징맵” 전체를 본다.**
3. **필터셋 개수 만큼** 특징맵을 만든다.

```
model = nn.Sequential( nn.Conv2d(in_channels=1, out_channels=3, kernel_size=5),  
nn.SiLU(),  
nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5), nn.SiLU(), )
```





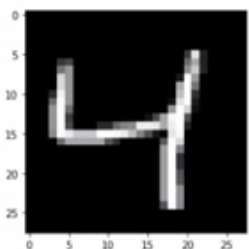
04. CNN핵심 기술-filter(필터)

filter(필터)

필터의 이해

1. 필터셋은 3차원 형태로 된 가중치의 모음
2. 필터셋 하나는 앞선 레이어의 결과인 “특징맵” 전체를 본다.
3. 필터셋 개수 만큼 특징맵을 만든다.

```
model = nn.Sequential( nn.Conv2d(in_channels=1, out_channels=3, kernel_size=5),  
nn.SiLU(),  
nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5), nn.SiLU(), )
```

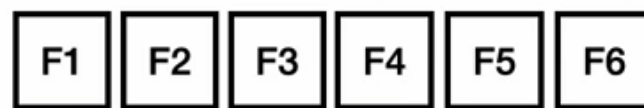


(28, 28, 1)



(5, 5, 1) ←

~~(3, 3, 5)~~



~~(6, 6, 5)~~



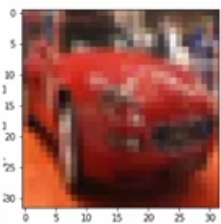
04. CNN핵심 기술-filter(필터)

■ filter(필터)

필터의 이해

1. 필터셋은 3차원 형태로 된 **가중치**의 모음
2. 필터셋 하나는 앞선 레이어의 결과인 **“특징맵”** 전체를 본다.
3. **필터셋 개수 만큼** 특징맵을 만든다.

```
model = nn.Sequential( nn.Conv2d(in_channels=1, out_channels=3, kernel_size=5),  
nn.SiLU(),  
nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5), nn.SiLU(), )
```

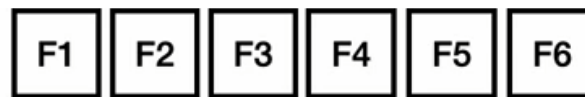


(32, 32, 3)



(5, 5, 3)

~~(3, 3, 5)~~



~~(6, 3, 5)~~



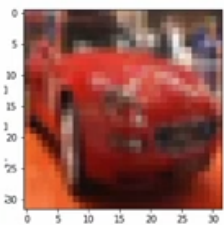
04. CNN핵심 기술-filter(필터)

■ filter(필터)

필터의 이해

1. 필터셋은 3차원 형태로 된 가중치의 모음
2. 필터셋 하나는 앞선 레이어의 결과인 “특징맵” 전체를 본다.
3. 필터셋 개수 만큼 특징맵을 만든다.

```
model = nn.Sequential( nn.Conv2d(in_channels=1, out_channels=3, kernel_size=5),  
nn.SiLU(),  
nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5), nn.SiLU(), )
```

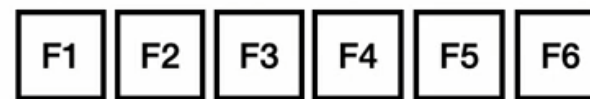


(32, 32, 3)



(5, 5, 3)

(3, 5, 5, ?)



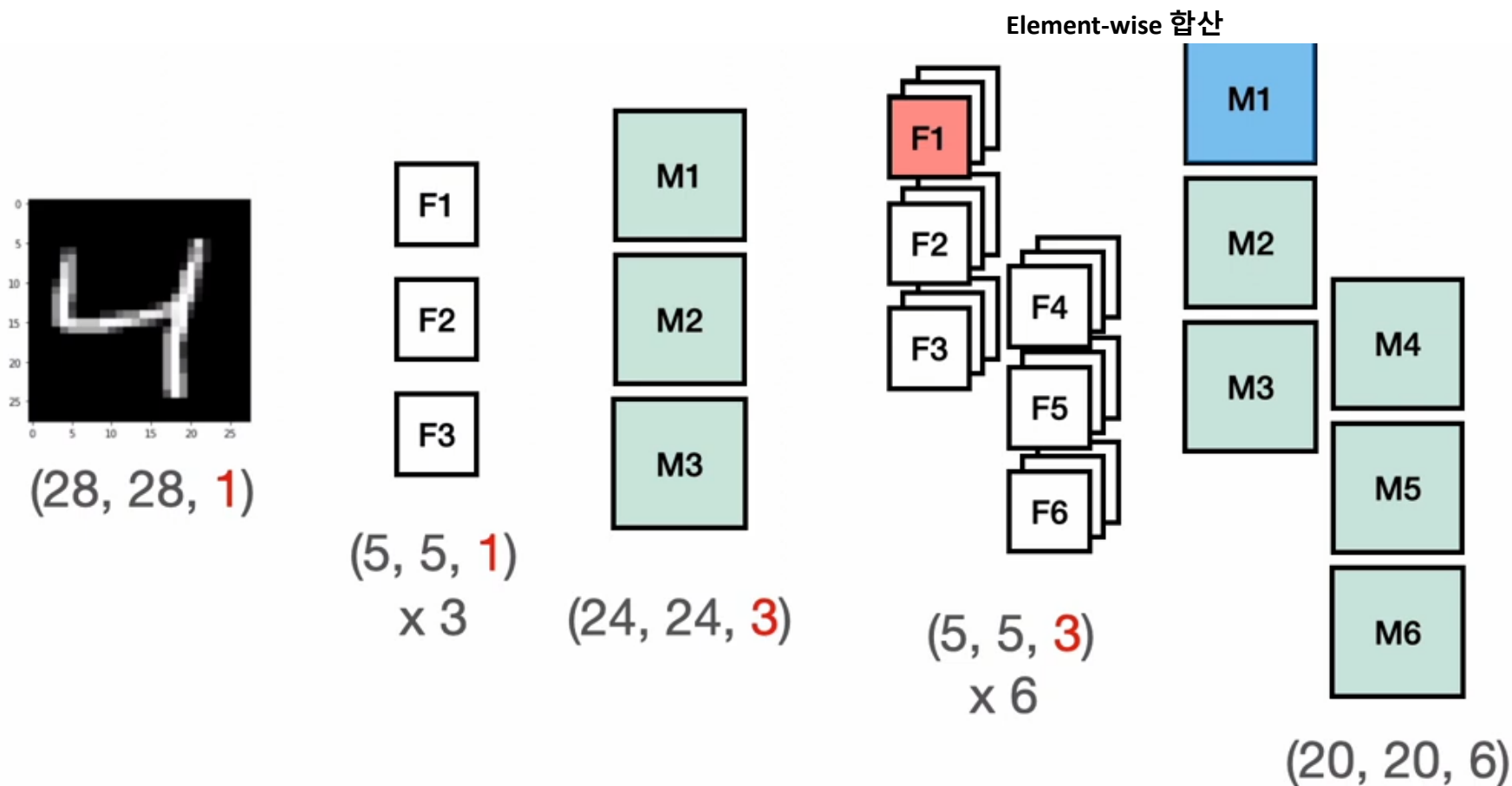
(6, 5, 5, ?)



04. CNN핵심 기술-filter(필터)

■ filter(필터)

```
model = nn.Sequential( nn.Conv2d(in_channels=1, out_channels=3, kernel_size=5),  
nn.SiLU(),  
nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5), nn.SiLU(), )
```





04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

x1	x2	x3	x4	x5	x6	x7	x8
x9	x10	x11	x12	x13	x14	x15	x16
x17	x18	x19	...				

(3, 3, 1)

w1	w2	w3
w4	w5	w6
w7	w8	w9

필터

(6, 6)

y1	y2	y3	y4	y5	y6
y7	y8	y9	y10	...	



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

x1	x2	x3	x4	x5	x6	x7	x8
x9	x10	x11	x12	x13	x14	x15	x16
x17	x18	x19	...				

(3, 3, 1)

w1	w2	w3
w4	w5	w6
w7	w8	w9

필터

(6, 6)

y1	y2	y3	y4	y5	y6
y7	y8	y9	y10	...	

$$\begin{aligned} y1 = & x1*w1 + x2*w2 + x3*w3 + \\ & x9*w4 + x10*w5 + x11*w6 + \\ & x17*w7 + x18*w8 + x19*w9 \end{aligned}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

x1	x2	x3	x4	x5	x6	x7	x8
x9	x10	x11	x12	x13	x14	x15	x16
x17	x18	x19	...				

(3, 3, 1)

w1	w2	w3
w4	w5	w6
w7	w8	w9

필터

(6, 6)

y1	y2	y3	y4	y5	y6
y7	y8	y9	y10	...	

$$y1 = x1*w1 + x2*w2 + x3*w3 + x9*w4 + x10*w5 + x11*w6 + x17*w7 + x18*w8 + x19*w9$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

x1	x2	x3	x4	x5	x6	x7	x8
x9	x10	x11	x12	x13	x14	x15	x16
x17	x18	x19	...				

(3, 3, 1)

w1	w2	w3
w4	w5	w6
w7	w8	w9

필터

(6, 6)

y1	y2	y3	y4	y5	y6
y7	y8	y9	y10	...	

$$y1 = x1*w1 + x2*w2 + x3*w3 + \\ x9*w4 + x10*w5 + x11*w6 + \\ x17*w7 + x18*w8 + x19*w9$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

x1	x2	x3	x4	x5	x6	x7	x8
x9	x10	x11	x12	x13	x14	x15	x16
x17	x18	x19	...				

(3, 3, 1)

w1	w2	w3
w4	w5	w6
w7	w8	w9

필터

(6, 6)

y1	y2	y3	y4	y5	y6
y7	y8	y9	y10	...	

$$y1 = x1*w1 + x2*w2 + x3*w3 + x9*w4 + x10*w5 + x11*w6 + x17*w7 + x18*w8 + x19*w9$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

$$\begin{aligned} = & \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \end{aligned}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1					

$$\begin{aligned} 1 = & 0 * -1 + 0 * -1 + 0 * -1 + \\ & 0 * 2 + 1 * 2 + 0 * 2 + \\ & 0 * -1 + 1 * -1 + 0 * -1 \end{aligned}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1					

$$\begin{aligned} = & 0 * -1 + 0 * -1 + 0 * -1 + \\ & 0 * 2 + 1 * 2 + 0 * 2 + \\ & 0 * -1 + 1 * -1 + 0 * -1 \end{aligned}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1					

$$\begin{aligned} 1 &= 0 * -1 + 0 * -1 + 0 * -1 + \\ &\quad 1 * 2 + 0 * 2 + 0 * 2 + \\ &\quad 1 * -1 + 0 * -1 + 0 * -1 \end{aligned}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1	1				

$$\begin{aligned} &= 0 * -1 + 0 * -1 + 0 * -1 + \\ &\quad 1 * 2 + 0 * 2 + 0 * 2 + \\ &\quad 1 * -1 + 0 * -1 + 0 * -1 \end{aligned}$$



04. CNN핵심 기술-convolution 연산

convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1	1	0			

$$\begin{aligned} Y = & 0 * -1 + 0 * -1 + 0 * -1 + \\ & 0 * 2 + 0 * 2 + 0 * 2 + \\ & 0 * -1 + 0 * -1 + 0 * -1 \end{aligned}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1	1	0	1		

$$\begin{aligned} y = & 0 * -1 + 0 * -1 + 0 * -1 + \\ & 0 * 2 + 0 * 2 + 1 * 2 + \\ & 0 * -1 + 0 * -1 + 1 * -1 \end{aligned}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1	1	0	1	1	1

$$y = \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix}$$



04. CNN핵심 기술-convolution 연산

convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1	1	0	1	1	1
0	0	0	0	0	0

$$y = \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터

(6, 6)

1	1	0	1	1	1
0	0	0	0	0	0
-1	-2	-3	-2	-2	-1
3	5	6	4	4	2
-2	-3	-3	-2	-2	-1
0	0	0	1	1	1

$$y = \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix}$$



04. CNN핵심 기술-convolution 연산

■ convolution 연산

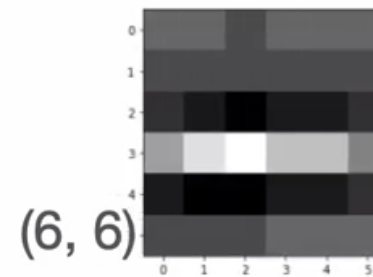
(8, 8, 1)

0	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

(3, 3, 1)

-1	-1	-1
2	2	2
-1	-1	-1

필터



$$y = \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix} \begin{matrix} * & -1 & + \\ * & 2 & + \\ * & -1 & + \end{matrix}$$

1	1	0	1	1	1
0	0	0	0	0	0
-1	-2	-3	-2	-2	-1
3	5	6	4	4	2
-2	-3	-3	-2	-2	-1
0	0	0	1	1	1



04. CNN핵심 기술-패딩(padding)

가장자리 정보 손실 문제

패딩 없이 합성곱을 수행하면 가장자리 픽셀이 거의 처리되지 않는다

5×5 입력 · 3×3 커널 · padding = 0

A	B	C	D	E
F	G	H	I	J
K	L	M	N	O
P	Q	R	S	T
U	V	W	X	Y

각 픽셀이 합성곱에 참여한 횟수

1	2	3	2	1	← 모서리: 1회
2	4	6	4	2	
3	6	9	6	3	← 중앙: 9회 (9배)
2	4	6	4	2	
1	2	3	2	1	← 모서리: 1회

문제 진단

불균등 처리: 모서리 픽셀(A, E, U, Y)은 단 1회만 처리되는 반면, 중앙(M)은 9회 처리됨

정보 손실: 가장자리 정보가 거의 무시됨 → 객체가 가장자리에 있으면 검출 실패 가능

크기 축소: 5×5 → 3×3으로 출력 크기 감소. 깊은 네트워크에서 누적되면 feature map 소실

사유: VGG, ResNet 등 대부분의 CNN이 padding을 사용하는 이유 — 위 3가지 문제 해결



04. CNN핵심 기술-패딩(padding)

0으로 둘러싸 균등 처리

padding=1을 적용하면 입력 주위에 0이 채워져 가장자리도 충분히 처리된다

패딩 적용 후 입력 (7×7)

0	0	0	0	0	0	0
0	A	B	C	D	E	0
0	F	G	H	I	J	0
0	K	L	M	N	O	0
0	P	Q	R	S	T	0
0	U	V	W	X	Y	0
0	0	0	0	0	0	0

각 픽셀이 합성곱에 참여한 횟수

4	6	6	6	4
6	9	9	9	6
6	9	9	9	6
6	9	9	9	6
4	6	6	6	4

→ 가장자리도 충분히 처리됨!

패딩의 3가지 효과

- 가장자리 정보 보존 (균등 처리)
- 출력 크기 조절 (입력 크기 유지 가능)
- 깊은 네트워크의 크기 급감 방지

패딩 크기 공식

패딩 후 크기 = 원본 + 2 × 패딩값

예) 28×28 , padding=1 → 30×30

예) 28×28 , padding=2 → 32×32

크기 유지 비법: padding = $(K-1) / 2$ [(원본+2p)-k+1=원본]



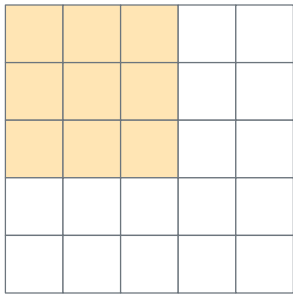
04. CNN핵심 기술-스트라이드(stride)

■ 커널의 이동 보폭

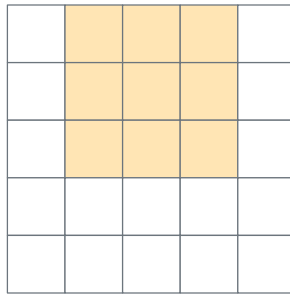
한 번에 몇 칸씩 움직일지 결정. 클수록 출력 크기가 작아진다

Stride = 1 (1칸씩 이동)

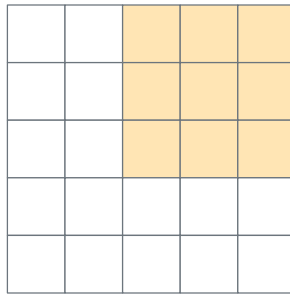
1번째



2번째



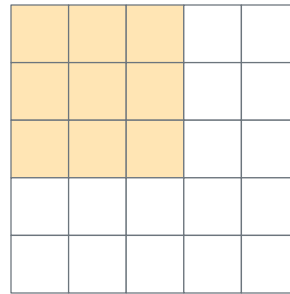
3번째



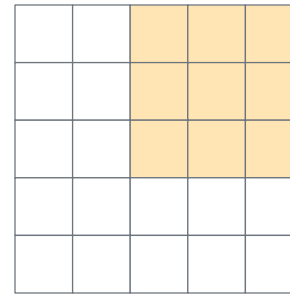
→ 출력 크기 큼 · 세밀한 특성

Stride = 2 (2칸씩 이동)

1번째



2번째 (2칸 점프)



→ 출력 크기 절반 · 다운샘플링

스트라이드별 출력 크기 (입력 224×224 , 커널 3×3 , padding=1)

Stride	출력 크기	비율	주 용도
1	224×224	$1 \times$	세밀한 특성 추출 (VGG, 일반 Conv)
2	112×112	$1/2 \times$	다운샘플링 (가장 흔함, ResNet)
4	56×56	$1/4 \times$	급격한 축소 (네트워크 첫 층)

Stride는 학습 파라미터가 아닌 하이퍼파라미터. MaxPool 없이 다운샘플링하는 효과가 있어 최근 모델에서 자주 사용됨.



04. CNN핵심 기술-확장(dilation)

■ 커널 원소 사이의 간격

커널 원소들 사이에 간격을 두어 같은 매개변수 수로 더 넓은 영역을 본다

Dilation = 1

수용영역(Receptive Field): 3×3

1	2	3
4	5	6
7	8	9

Dilation = 2

수용영역(Receptive Field): 5×5

1		2		3
4		5		6
7		8		9

Dilation = 3

수용영역(Receptive Field): 7×7

1		2		3		
4		5		6		
7		8		9		

매개변수 (학습 가중치)

항상 9개 (3×3)

Dilation은 학습할 파라미터를 늘리지 않는다

수용영역 (Receptive Field)

$3 \times 3 \rightarrow 5 \times 5 \rightarrow 7 \times 7$

파라미터 증가 없이 시야만 약 2.3배 확대

실제 활용

의료 영상 (병변+장기)
자율주행 (차선+신호등)
위성 영상 (디테일+전체)



04. CNN핵심 기술-확장(dilation)

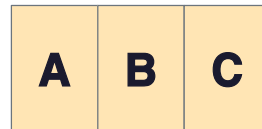
▣ 1차원 예시로 직관 잡기

1D 커널 [A][B][C]가 dilation 값에 따라 어떻게 펼쳐지는지 확인

dilation = 1 (일반)

실제 크기: 3

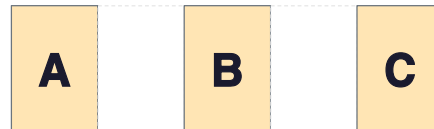
매개변수: 3개



dilation = 2

실제 크기: 5

매개변수: 3개



dilation = 3

실제 크기: 7

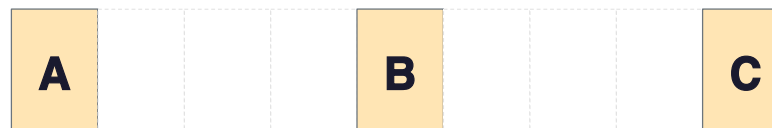
매개변수: 3개



dilation = 4

실제 크기: 9

매개변수: 3개





04. CNN핵심 기술-확장(dilation)

▣ 2D 3×3 커널 비교

dilation 값이 커질수록 같은 9개 원소가 더 넓은 영역을 커버

Dilation = 1

실제 크기: 3×3

1	2	3
4	5	6
7	8	9

Dilation = 2

실제 크기: 5×5

1		2		3
4		5		6
7		8		9

Dilation = 3

실제 크기: 7×7

1			2			3
4			5			6
7			8			9

Dilation = 4

실제 크기: 9×9

1				2				3
4				5				6
7				8				9

확장 커널 크기 공식

$$K_{\text{eff}} = K_{\text{orig}} + (D - 1) \times (K_{\text{orig}} - 1)$$

$$3 \times 3 + D=2 \rightarrow 3 + (2-1) \times 2 = 5 \quad \cdot \quad 3 \times 3 + D=3 \rightarrow 3 + (3-1) \times 2 = 7 \quad \cdot \quad 3 \times 3 + D=4 \rightarrow 3 + (4-1) \times 2 = 9$$



04. CNN핵심 기술-확장(dilation)

▣ 다른 커널 크기 & 비대칭 확장

커널 크기가 다르거나 세로 · 가로 dilation이 다를 때의 확장 패턴

5×5 커널

dilation = 1 (실제 5×5)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

dilation = 2 (실제 9×9)

1	2	3	4	5				
6	7	8	9	10				
11	12	13	14	15				
16	17	18	19	20				
21	22	23	24	25				

비대칭 확장: 3×5 커널 · dilation = (2, 3)

세로 dilation 2 · 가로 dilation 3

1			2			3			4			5
6			7			8			9			10
11			12			13			14			15

실제 크기: 5 × 13 (세로 5 = 3 + (2-1)×2, 가로 13 = 5 + (3-1)×4)

일반 공식 (모든 커널 크기에 적용)

$$K_{\text{effective}} = K_{\text{original}} + (\text{Dilation} - 1) \times (K_{\text{original}} - 1)$$

3×3, D=2

$$3 + 1 \times 2 = 5$$

3×3, D=3

$$3 + 2 \times 2 = 7$$

5×5, D=2

$$5 + 1 \times 4 = 9$$

7×7, D=3

$$7 + 2 \times 6 = 19$$



04. CNN핵심 기술-확장(dilation)

■ 실제 활용 사례

한 모델에서 dilation을 다르게 조합하여 다양한 스케일의 패턴을 동시 학습

의료 영상 분석

D=1

세포 경계선 감지 (미세 구조)

D=3

조직 구조 파악 (중간 패턴)

D=5

장기 전체 형태 인식 (광역)

자율주행 차량

D=1

차선 마킹 감지 (가까운 패턴)

D=2

인접 차량 인식 (중거리)

D=4

신호등 · 표지판 검출 (원거리)

위성 이미지 분석

D=1

건물 모서리 감지 (디테일)

D=3

도로 네트워크 파악 (중간)

D=5

도시 전체 레이아웃 (광역)

Dilation의 핵심 가치

같은 파라미터 수로 더 넓은 영역 관찰 · 다양한 스케일의 특성을 효율적으로 포착 · 멀리 떨어진 픽셀들 간의 관계 학습 가능
대표 모델: DeepLab v2/v3 (Atrous Convolution), WaveNet (오디오 dilation), TCN



04. CNN핵심 기술-확장(dilation)

▣ 실제 활용 사례

비대칭 확장은 특수 목적용 기법으로, 데이터의 축 별 정보 밀도가 다를 때 사용함

도메인	H축 (세로)	W축 (가로)	적합한 Dilation
도로 차선 검출	변화 느낌	멀리까지 연결	$D=(1, 4)$
텍스트 OCR	글자 한 줄 높이	단어 · 문장 길이	$D=(1, 3\sim 5)$
시계열 + 채널	채널 수 (적음)	시간축 (길이)	$D=(1, N)$
의료 CT (axial)	슬라이스 간격 큼	픽셀 해상도 높음	$D=(3, 1)$
위성 영상 (스캔라인)	스캔 방향	비행 방향	$D=(2, 1)$



04. CNN핵심 기술-MaxPool2D

■ MaxPool2D

10	7	2	42	30	25
1	16	33	8	7	6
9	22	11	45	10	22
13	15	31	27	12	41
7	20	19	30	23	6
3	24	15	27	14	40



04. CNN핵심 기술-MaxPool2D

■ MaxPool2D

10	7	2	42	30	25
1	16	33	8	7	6
9	22	11	45	10	22
13	15	31	27	12	41
7	20	19	30	23	6
3	24	15	27	14	40

16		



04. CNN핵심 기술-MaxPool2D

■ MaxPool2D

10	7	2	42	30	25
1	16	33	8	7	6
9	22	11	45	10	22
13	15	31	27	12	41
7	20	19	30	23	6
3	24	15	27	14	40

16	42	



04. CNN핵심 기술-MaxPool2D

■ MaxPool2D

10	7	2	42	30	25
1	16	33	8	7	6
9	22	11	45	10	22
13	15	31	27	12	41
7	20	19	30	23	6
3	24	15	27	14	40

16	42	30



04. CNN핵심 기술-MaxPool2D

■ MaxPool2D

10	7	2	42	30	25
1	16	33	8	7	6
9	22	11	45	10	22
13	15	31	27	12	41
7	20	19	30	23	6
3	24	15	27	14	40

16	42	30
22		



04. CNN핵심 기술-MaxPool2D

■ MaxPool2D

10	7	2	42	30	25
1	16	33	8	7	6
9	22	11	45	10	22
13	15	31	27	12	41
7	20	19	30	23	6
3	24	15	27	14	40

MaxPooling

16	42	30
22	45	41
24	30	40



04. CNN핵심 기술-MaxPool2D

■ MaxPool2D

나는 특징맵

10	7	2	42	30	25
1	16	33	8	7	6
9	22	11	45	10	22
13	15	31	27	12	41
7	20	19	30	23	6
3	24	15	27	14	40

MaxPooling

16	42	30
22	45	41
24	30	40

값이 크다 = 필터로 찾으려는 특징이 많이 나타난 부분



05. CNN 출력 계산

▣ 출력 크기 계산 · 통합 공식

패딩 · 스트라이드 · 확장을 모두 반영한 일반 공식

출력 크기 (한 번 기준)

$$\text{Output} = \left[\frac{(I + 2P - K_{\text{eff}})}{S} \right] + 1$$

$K_{\text{eff}} = K + (D-1) \times (K-1)$: 확장된 커널 크기

4단계 계산법

확장된 커널 크기

$$K_{\text{eff}} = K + (D-1) \times (K-1)$$

패딩 후 입력 크기

$$I_{\text{pad}} = I + 2P$$

커널 적용 유효 범위

$$V = I_{\text{pad}} - K_{\text{eff}}$$

최종 출력 크기

$$\text{Output} = \left[\frac{V}{S} \right] + 1$$

변수 정의

I 입력 크기 **K** 커널 크기 **P** 패딩값 **S** 스트라이드 **D** 확장값 **[·]** 내림(floor)

세로(H)와 가로(W)는 서로 독립적으로 계산. 비대칭 파라미터(예: $K=3 \times 5$)도 각 축마다 따로 적용.



05. CNN 출력 계산

■ 계산 예시 · 50×100 입력 (비대칭 파라미터)

세로와 가로가 서로 다른 파라미터를 가질 때 각 축을 독립적으로 계산

주어진 조건

입력 50×100 · 커널 3×5 · 패딩 4×2 · 스트라이드 2×1 · 확장 3×1 (모든 값: 세로 $\times$ 가로)

세로 방향 (Height)

- ① 확장된 커널: $K_{\text{eff}} = 3 + (3-1) \times (3-1) = 7$
- ② 패딩 후 크기: $I_{\text{pad}} = 50 + 2 \times 4 = 58$
- ③ 유효 범위: $V = 58 - 7 = 51$
- ④ 최종 크기: $[51/2] + 1 = 25 + 1 = 26$

세로 결과 = 26

가로 방향 (Width)

- ① 확장된 커널: $K_{\text{eff}} = 5 + (1-1) \times (5-1) = 5$
- ② 패딩 후 크기: $I_{\text{pad}} = 100 + 2 \times 2 = 104$
- ③ 유효 범위: $V = 104 - 5 = 99$
- ④ 최종 크기: $[99/1] + 1 = 99 + 1 = 100$

가로 결과 = 100

최종 출력 크기: $50 \times 100 \rightarrow 26 \times 100$

05. CNN 출력 계산

■ 전체 4D 텐서 관점에서의 변환

배치 · 채널 · 공간 차원을 모두 포함한 입출력 shape 변화



쉬운 기억법

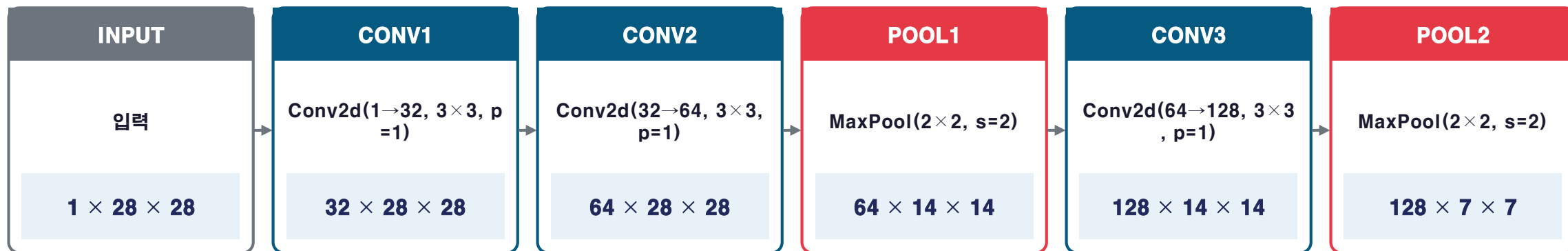
패딩으로 집을 크게 만들고 커널만큼 잘라내고 스트라이드만큼 건너뛰며 세고 **마지막에 1을 더한다**



06. CNN 예시

■ 실전 예시 · MNIST 손글씨 인식 CNN

28×28 흑백 이미지 → 0~9 분류. padding=1로 크기 유지, MaxPool로 다운샘플링



크기 변화 추적 (공식 검증)

단계	입력	파라미터	공식 적용	출력
Conv1	28×28	$K=3$, $P=1$, $S=1$	$(28 + 2 - 3)/1 + 1 = 28$	28×28
Conv2	28×28	$K=3$, $P=1$, $S=1$	$(28 + 2 - 3)/1 + 1 = 28$	28×28
Pool1	28×28	$K=2$, $P=0$, $S=2$	$(28 - 2)/2 + 1 = 14$	14×14
Conv3	14×14	$K=3$, $P=1$, $S=1$	$(14 + 2 - 3)/1 + 1 = 14$	14×14
Pool2	14×14	$K=2$, $P=0$, $S=2$	$(14 - 2)/2 + 1 = 7$	7×7

패턴: Conv는 padding=1로 크기 유지, MaxPool이 다운샘플링 담당. 채널은 $32 \rightarrow 64 \rightarrow 128$ 로 증가 (특성 표현력 강화)

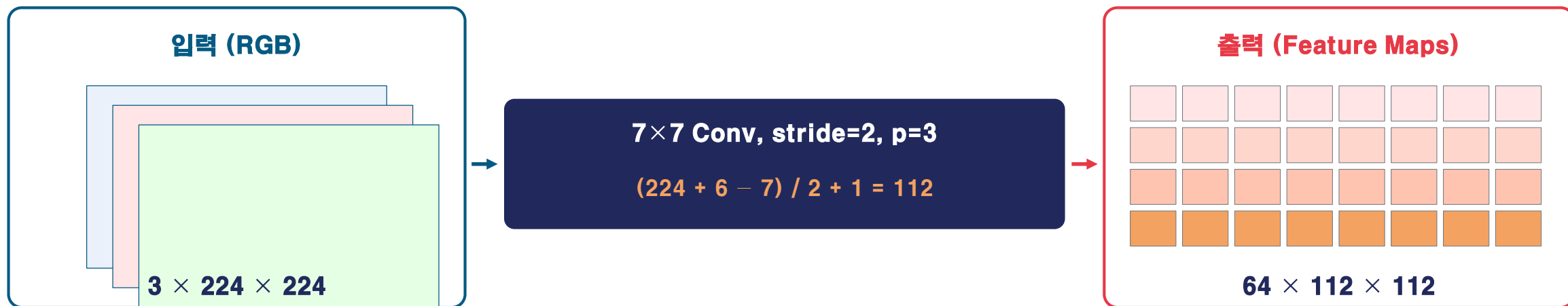
06. CNN 예시

■ 실전 예시 · ImageNet (ResNet 첫 층)

224×224 컬러 이미지를 7×7 큰 커널 + stride=2로 한 번에 절반 축소

```
nn.Conv2d(in_channels=3, out_channels=64, kernel_size=7, stride=2, padding=3)
```

RGB 이미지 → 저수준 특성맵 64장 (7×7 큰 커널로 광범위 시야 확보)



설계 의도 분석

큰 커널 7×7 — 첫 층에서 광범위한 저수준 특성(엣지, 색 변화, 텍스처) 검출

Stride = 2 — 메모리 · 연산량 절약 위해 즉시 절반 다운샘플링 (이후 layer 부담 감소)

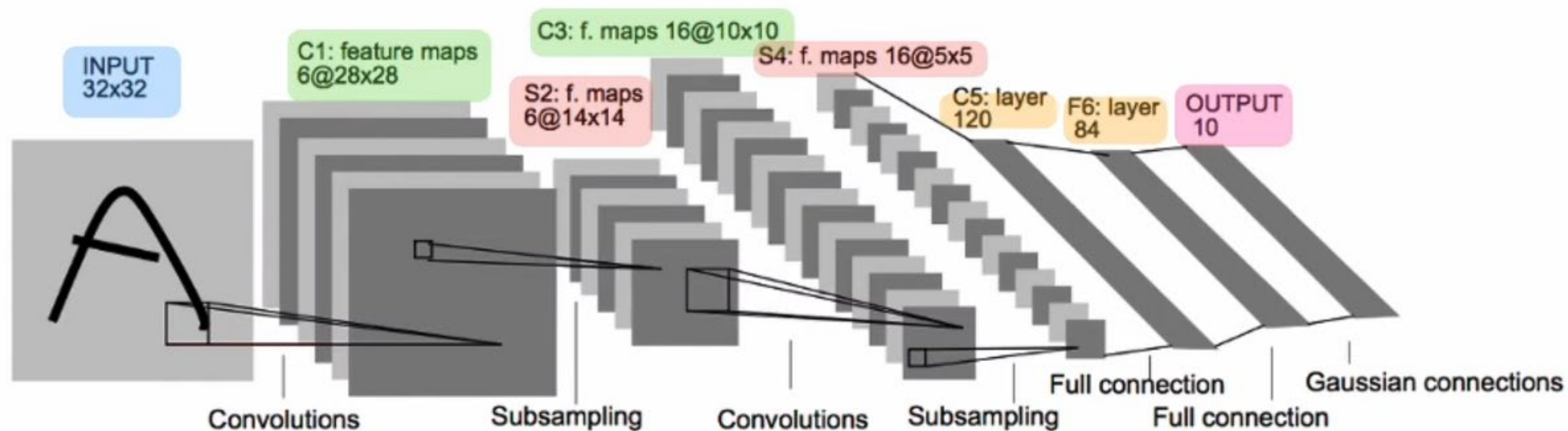
Padding = 3 — 정확히 절반(112)이 나오도록 조정. $(K-1)/2 = (7-1)/2 = 3$

Channels: 3 → 64 — RGB 3채널을 64개의 다양한 저수준 특징 채널로 확장



05. CNN 예시

■ 실전 예시 · LeNet 5



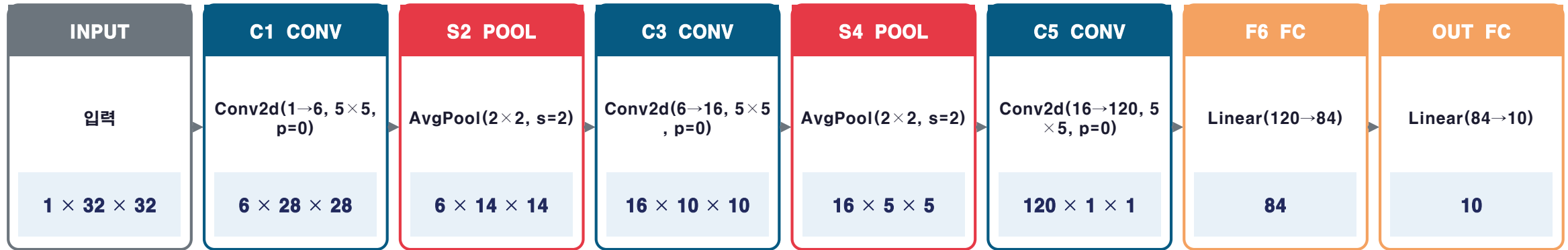
CNN called LeNet by Yann LeCun (1998)



05. CNN 예시

실전 예시 · LeNet 5

32×32 흑백 이미지 → 0~9 분류. CNN의 원조 — padding 없이 매 Conv마다 크기 축소



크기 변화 추적 (공식 검증)

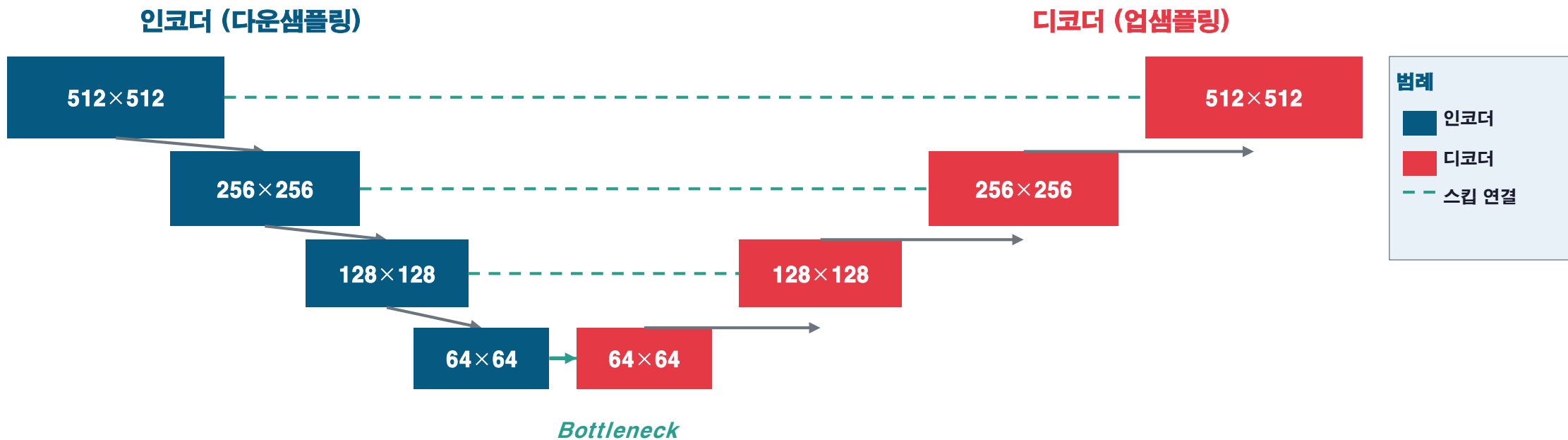
단계	입력	파라미터	공식 적용	출력
C1 (Conv)	32×32	K=5, P=0, S=1	$(32 + 0 - 5)/1 + 1 = 28$	28×28
S2 (Pool)	28×28	K=2, P=0, S=2	$(28 - 2)/2 + 1 = 14$	14×14
C3 (Conv)	14×14	K=5, P=0, S=1	$(14 + 0 - 5)/1 + 1 = 10$	10×10
S4 (Pool)	10×10	K=2, P=0, S=2	$(10 - 2)/2 + 1 = 5$	5×5
C5 (Conv)	5×5	K=5, P=0, S=1	$(5 + 0 - 5)/1 + 1 = 1$	1×1 (120ch)
F6 (FC)	120	Linear	120 → 84	84
Out (FC)	84	Linear + Softmax	84 → 10	10 (클래스)

패턴: LeNet-5는 padding을 쓰지 않음 → Conv마다 (K-1)=4픽셀씩 크기 감소 (32→28→14→10→5→1). 채널: 1 → 6 → 16 → 120 → 84 → 10

06. CNN 예시

실전 예시 · U-Net 의료 영상 분할

512×512 영상을 인코더로 압축 후 디코더로 복원. 스킵 연결로 세밀 정보 보존



인코더 (압축)

Stride=2 Conv 또는 MaxPool로 공간 해상도 절반씩 축소. 의미 정보를 추출하며 채널 수는 증가 (16→32→64→128).

디코더 (복원)

Transposed Conv 또는 Upsample로 공간 해상도를 두 배씩 확대. 픽셀별 분류 수행 (Semantic Segmentation).

스킵 연결 (Skip)

인코더의 세밀한 정보를 디코더로 직접 전달. 압축 과정에서 손실된 디테일을 복원. 정밀한 경계 검출의 핵심.